



UNIVERSIDAD CONTINENTAL
FACULTAD DE INGENIERÍA
Escuela Académico Profesional de Ingeniería de Sistemas e Informática

PROYECTO DE INVESTIGACIÓN
"Ciclo de vida de desarrollo de Software con la metodología SDLC"

PRESENTADO POR:
Rojas Camayo Valentino Jhan Pierre

Para optar el Título Profesional de
Ingeniero de Sistemas e Informática

ASESOR:
Dr. Maglioni Arana Caparachin

Huancayo – Perú
2026

Contenido

CAPÍTULO 1: EL PROBLEMA	6
1.1 Situación Problemática.....	6
1.2 Antecedentes del Problema	11
1.3 Formulación del Problema.....	13
1.4 Justificación de la Investigación	15
1.4.1 Justificación Teórica	15
1.4.2 Justificación Práctica.....	16
1.4.3 Justificación Metodológica	16
1.4.4 Justificación Social y Pertinencia Local.....	16
1.5 Objetivos de la Investigación.....	17
Objetivo General	17
Objetivos Específicos	17
1.6 Delimitación de la Investigación.....	18
CAPÍTULO 2: MARCO TEÓRICO Y ESTADO DEL ARTE.....	18
2.1 Introducción al Capítulo.....	18
2.2 Marco Teórico	19
2.2.1 El Ciclo de Vida del Desarrollo de Software (SDLC).....	19
2.2.2 La Inteligencia Artificial Generativa y los LLMs	19
2.2.3 La Ingeniería del Prompt.....	20
2.2.4 Los Modelos AI-Native del SDLC	21
2.3 Estado del Arte por Dimensiones	21
2.3.1 Dimensión 1: Diseño e Ingeniería del Prompt para el SDLC.....	22
2.3.2 Dimensión 2: Evaluación, Refinamiento y Gestión del Prompt.....	23
2.3.3 Dimensión 3: El Prompt como Habilitador de las Fases del SDLC	23
2.3.4 Dimensión 4: Automatización Agéntica y Pipelines IA en el SDLC	25
2.3.5 Dimensión 5: Seguridad, Gobernanza y Monitoreo del Ciclo AI-Native.....	26
2.4 Síntesis del Estado del Arte: Brechas que Justifican el Modelo Propuesto	27
CAPÍTULO 3: METODOLOGÍA DE LA INVESTIGACIÓN	28
3.1 Enfoque de la Investigación.....	28
3.2 Tipo y Nivel de la Investigación	29
3.2.1 Tipo de Investigación.....	29

3.2.2 Nivel de la Investigación.....	29
3.3 Diseño de la Investigación.....	29
3.3.1 Cómo el Análisis del Estado del Arte Dio Origen al Modelo Propuesto.....	30
3.4 Población y Unidad de Análisis	31
3.5 Técnicas e Instrumentos de Recolección de Datos	31
3.5.1 Revisión Sistemática de Literatura.....	31
3.5.2 Análisis Documental.....	32
3.5.3 Matriz de Operabilidad de Variables	32
3.6 Variables de la Investigación.....	32
3.6.1 Variable Independiente.....	32
3.6.2 Variable Dependiente	32
3.7 Matriz de Operabilidad de Variables.....	33
3.8 Procedimiento de la Investigación.....	34
CAPÍTULO 4: GESTIÓN ADMINISTRATIVA.....	35
4.1 Introducción al Capítulo.....	35
4.2 Cronograma de Actividades.....	35
4.2.1 Fase I — Elaboración del Plan de Tesis	35
4.2.2 Fase II — Desarrollo de la Tesis Completa.....	36
4.3 Presupuesto de la Investigación	37
4.3.1 Recursos Humanos	37
4.3.2 Recursos Materiales	38
4.3.3 Recursos Tecnológicos	38
4.3.4 Servicios y Trámites	39
4.3.5 Resumen del Presupuesto Total.....	40
4.4 Financiamiento.....	40
CAPÍTULO 5: RESULTADOS DE LA INVESTIGACIÓN	40
5.1 Introducción al Capítulo.....	40
5.2 Resultados del Análisis Sistemático de Literatura.....	41
5.2.1 Características del Corpus Documental.....	41
5.2.2 Distribución de Fuentes por Dimensión del Modelo	41
5.2.3 Brechas Identificadas por Dimensión	42
5.3 Análisis Comparativo del Modelo Propuesto.....	43

5.3.1 Criterios de Comparación	43
5.3.2 Tabla Comparativa.....	44
5.3.3 Interpretación del Análisis Comparativo	45
5.3.4 Posicionamiento del Modelo en el Campo	46
5.4 El Modelo Prompt-Centered SDLC como Resultado Consolidado	47
5.4.1 Fundamentación del Modelo	47
5.4.2 Arquitectura General del Modelo.....	48
5.4.3 D1 — Diseño e Ingeniería del Prompt.....	49
5.4.4 D2 — Evaluación, Refinamiento y Gestión del Prompt	51
5.4.5 D3 — El Prompt en Cada Fase del SDLC.....	52
5.4.6 D4 — Automatización Agéntica.....	54
5.4.7 D5 — Seguridad, Gobernanza y Monitoreo.....	56
5.4.8 Síntesis del Modelo — Cuatro Aspectos Diferenciadores.....	57
5.4.9 Pertinencia Latinoamericana y Peruana	58
INTERVENCIÓN METODOLÓGICA	58
IM.1 Introducción.....	58
IM.2 Descripción del Caso de Demostración	59
IM.2.1 Contexto del Sistema	59
IM.2.2 Objetivo de la Demostración	59
IM.3 Aplicación del Modelo por Fase del SDLC	60
IM.3.1 Fase 1 — Requisitos	60
IM.3.2 Fase 2 — Diseño.....	63
IM.3.3 Fase 3 — Implementación	65
IM.3.4 Fase 4 — Testing.....	68
IM.3.5 Fase 5 — Mantenimiento.....	70
IM.4 Síntesis de la Demostración	72
IM.4.1 Inventario de Prompts Generados.....	72
IM.4.2 Activación de Dimensiones por Fase	73
IM.4.3 Hallazgos de la Demostración.....	74
IM.4.4 Conclusión de la Intervención	75
ANEXOS.....	75
Índice de Anexos.....	75

Anexo A — Ficha de Extracción para Revisión Sistemática de Literatura.....	75
Ficha de Extracción RSL — Formato Estándar	76
Ejemplo de Ficha Completada	77
Anexo B — Matriz de Operacionalización de Variables (versión expandida).....	78
Variable Independiente: Ingeniería del Prompt en el SDLC	78
Variable Dependiente: Eficiencia del Proceso de Desarrollo de Software	79
Anexo C — Tabla Maestra de las 28 Referencias del Corpus Documental	80
Anexo D — Catálogo de Prompts del Modelo Prompt-Centered SDLC.....	84
Estructura de cada entrada del catálogo	84
Resumen del Catálogo	85
Anexo E — Glosario de Términos Técnicos	87
Anexo F — Diagrama del Modelo Prompt-Centered SDLC.....	89

CAPÍTULO 1: EL PROBLEMA

1.1 Situación Problemática

El desarrollo de software es, desde sus orígenes, una actividad intensiva en conocimiento humano que ha generado valor económico y social incalculable. Sin embargo, también es una de las disciplinas de ingeniería con las tasas de fracaso más altas y consistentes documentadas en la historia de la gestión de proyectos. El Standish Group, en su CHAOS Report 2020 —el estudio más extenso del campo, basado en el análisis de más de 50,000 proyectos tecnológicos a nivel global—, concluye que el 66% de los proyectos de software terminan en fracaso parcial o total [29]. De ese universo, el 31% de los proyectos son cancelados antes de completarse, y el 53% enfrenta desafíos tan severos de tiempo, costo o alcance que se catalogan como "problemáticos" [29]. Los proyectos de gran escala presentan los peores resultados: menos del 10% de los proyectos grandes concluye exitosamente sin desviaciones significativas [29]. Estos datos no representan una anomalía puntual: son el resultado de 25 años de seguimiento sistemático que revelan que el fracaso en el desarrollo de software no es la excepción, sino la norma estadística [29].

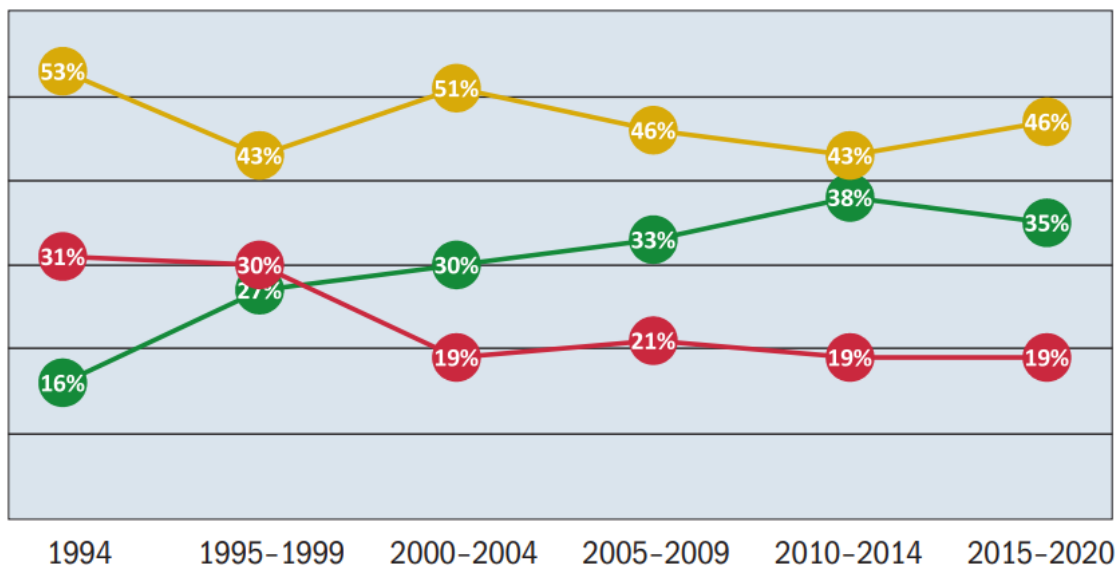


Figura 1. Evolución de las tasas de éxito, desafío y fracaso de proyectos de software (1994–2020).

Fuente: The Standish Group.

El impacto económico de esta realidad es de proporciones extraordinarias. McKinsey & Company, en un análisis desarrollado en colaboración con la Universidad de Oxford sobre más de 500 proyectos de gran escala —cada uno con presupuesto inicial superior a \$15 millones de dólares—, encontró que los proyectos de TI presentan en promedio un sobre costo del 45% respecto al presupuesto original y entregan un 56% menos de valor del proyectado [30]. El mismo análisis reveló que el 17% de los proyectos de TI llega a amenazar directamente la viabilidad financiera de las organizaciones que los desarrollan [30]. Para 2022, el Consortium for Information and Software Quality (CISQ) cuantificó el impacto directo de esta realidad: el costo total de la mala calidad del software en Estados Unidos alcanzó \$2.41 billones de dólares, mientras que la deuda técnica acumulada —el costo de retrabajar software de calidad subóptima— llegó a \$1.52 billones de dólares adicionales [31]. A escala global, el 70% de los proyectos de software reporta sobre costos, con una desviación promedio del 27% sobre el presupuesto inicial [32].

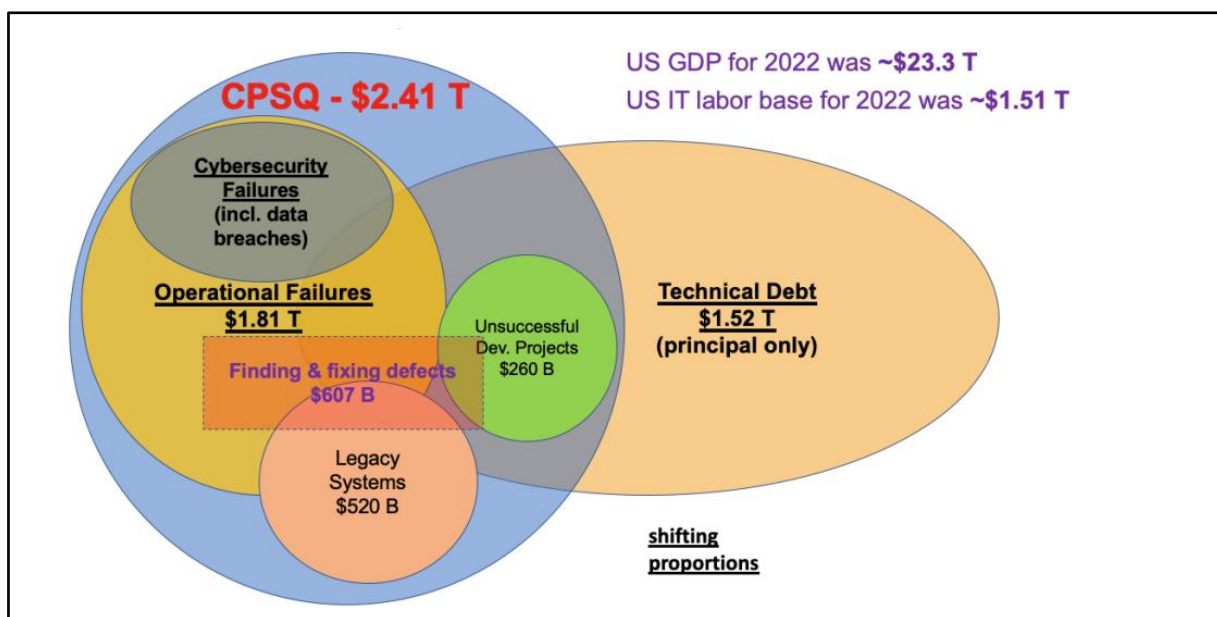


Figura 3. Costo de la mala calidad del software en Estados Unidos (2022). Fuente: (CISQ 2022).

La búsqueda de soluciones a este problema ha dado origen a las diferentes metodologías del ciclo de vida del desarrollo de software (SDLC). El modelo Waterfall estableció la primera respuesta estructurada: una secuencia lineal de

fases —Requisitos, Diseño, Implementación, Verificación y Mantenimiento— que prometía orden, trazabilidad y control [1][2]. Sin embargo, su rigidez ante el cambio lo convirtió en un modelo inadecuado para la realidad dinámica del desarrollo moderno. Las metodologías Agile surgieron como respuesta directa: Scrum, Kanban y Extreme Programming priorizan la entrega incremental, la colaboración con el cliente y la adaptación continua [3][4]. La revisión sistemática de Aleryani, Alsabry y Alramada confirma que Agile ha generado mejoras verificables en la satisfacción del cliente y en la velocidad de entrega respecto a Waterfall [4]. El CHAOS Report 2020 cuantifica esta diferencia: los proyectos Agile tienen tres veces más probabilidad de éxito que los proyectos Waterfall, y Waterfall presenta el doble de probabilidad de fracaso total [29][33].

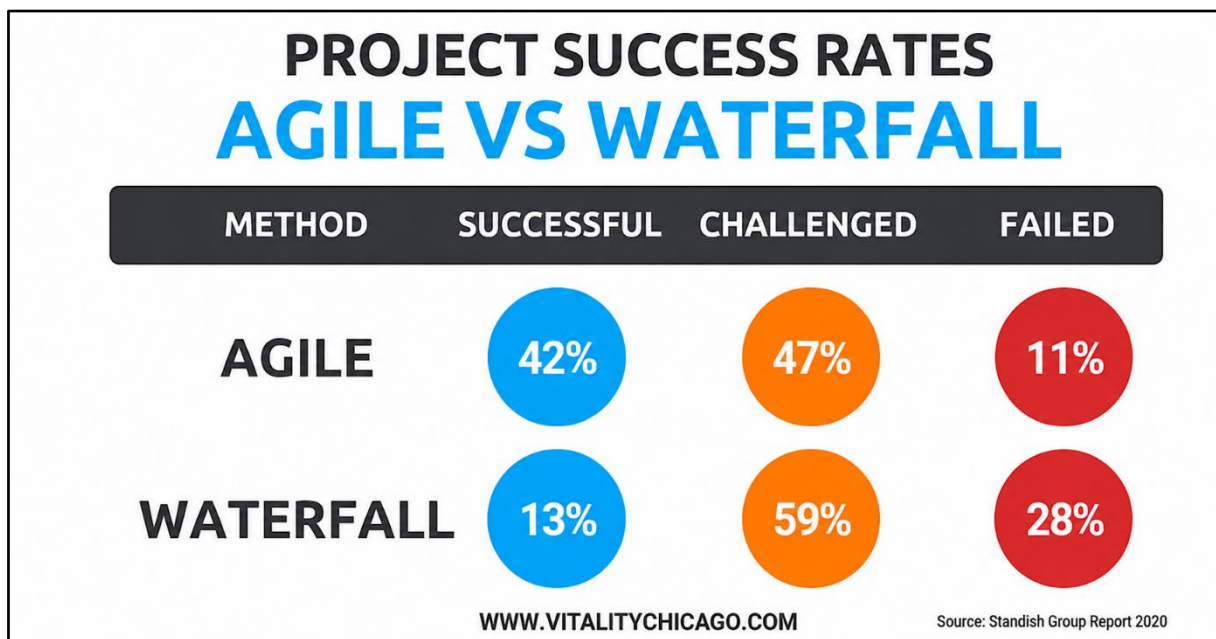


Figura 2. Comparativa de resultados de proyectos Agile vs. Waterfall. Fuente: The Standish Group.

Sin embargo, el avance de Agile no ha resuelto el problema de fondo. La revisión de Yas, ALazzawi y Rahmatullah sobre los principales modelos SDLC —Waterfall, Espiral, Agile, Scrum, Kanban— concluye que ninguno ha logrado resolver de manera satisfactoria la ecuación entre velocidad, calidad y costo [3]. La revisión de Aleryani et al. identifica limitaciones persistentes en Agile: la gestión del conocimiento en equipos distribuidos, la dificultad de escalar en proyectos grandes y la ausencia de métricas estandarizadas de calidad continúan sin

resolverse [4]. La evidencia de Tetteh agrega datos concretos: los equipos Scrum reportan una reducción del 37% en tiempos de entrega respecto a Waterfall, pero enfrentan un incremento del 28% en el esfuerzo de coordinación cuando el equipo supera los 10 integrantes [26]. El mantenimiento de software —frecuentemente subestimado— representa entre el 60% y el 80% del costo total del ciclo de vida de un sistema [24], y ninguna metodología existente ha propuesto una solución sistémica a este componente dominante del costo. En otras palabras, las metodologías mejoran el proceso, pero no lo transforman: el problema estructural de la dependencia exclusiva del desarrollador humano como actor de ejecución permanece intacto.

Es en este contexto de déficit metodológico crónico donde la Inteligencia Artificial Generativa (IAG) emerge como la tecnología con mayor potencial disruptivo jamás introducida en el campo. La revisión sistemática de Hou et al. —que analiza 395 estudios— y la de Zhang et al. —con 1009 papers— establecen con solidez empírica que los LLMs tienen capacidad de asistir, acelerar o automatizar prácticamente todas las fases del ciclo de vida del software [7][8]. El estudio comparativo de Abdalhamid y Almabruk cuantifica el impacto: la integración estructurada de IA en el SDLC produce una aceleración del 40% en los ciclos de desarrollo y una reducción del 60% en el tiempo de elicitación de requisitos, mientras que herramientas como GitHub Copilot aceleran la codificación en un 35% [9].

SDLC Phase	Traditional Methods	AI-Driven Techniques	Comparison
Requirements Gathering	- Manual stakeholder interviews and workshops. - Time-consuming and prone to errors.	- NLP extracts requirements from communication logs. - Faster and scalable.	Effectiveness: AI is faster but may lack depth. Quality: AI ensures consistency but may misinterpret nuances.
Design	- Manual brainstorming and design creation. - Relies on human creativity.	- Generative design and ML suggest templates. - Automates repetitive tasks.	Effectiveness: AI speeds up design but lacks creativity. Quality: AI improves consistency but may miss details.
Implementation	- Manual coding and debugging. - Prone to human error.	- AI tools (e.g., GitHub Copilot) automate coding and debugging. - Reduces errors.	Effectiveness: AI speeds up coding but requires human validation. Quality: AI improves code readability and reduces bugs.
Testing	- Manual testing is labor-intensive. - Limited test coverage.	- AI automates test case generation and defect detection. - Comprehensive coverage.	Effectiveness: AI is faster and more scalable. Quality: AI improves coverage but may miss edge cases.
Maintenance	- Reactive issue resolution based on user feedback. - Resource-intensive.	- Predictive maintenance anticipates failures. - Proactive and efficient.	Effectiveness: AI reduces downtime. Quality: AI improves reliability but requires human oversight.

Figura 4. Impacto de la integración de IA en el SDLC: beneficios y riesgos documentados. Fuente: Abdalhamid y Almabruk.

Sin embargo, esta capacidad transformadora viene acompañada de nuevos riesgos críticos. El código producido por IA presenta hasta un 25% más de vulnerabilidades de seguridad cuando no existe un proceso de revisión humana estructurado, y la usabilidad de los diseños generados puede reducirse hasta en un 20% sin criterios explícitos de validación [9]. La mayoría de las aproximaciones actuales trata a la IA como una herramienta que se añade al SDLC existente sin replantear el ciclo mismo [3][4]. El mapeo bibliométrico de Navaei y Tabrizi —que analizó 150 artículos en ACM, IEEE y Springer— evidenció que la integración de IA en el SDLC es fragmentada, reactiva y no sistémica: cada investigación aborda una fase de forma aislada sin proponer un modelo integral que conecte todas las etapas bajo una lógica unificada [6]. En el contexto latinoamericano, Paladino y Pons confirman que los principales obstáculos para integrar IA en el SDLC no son tecnológicos sino metodológicos: ausencia de procesos estructurados, falta de competencias en ingeniería de prompts y desconocimiento de los riesgos de

seguridad del código generado por IA [14]. En el contexto peruano específicamente, Rivas Verastegui, Tirado Ruiz y Torres Villanueva documentan que la adopción de herramientas como GitHub Copilot ya es una realidad en empresas y universidades del país, pero que los programadores novatos generan código significativamente menos seguro cuando no existe un proceso de supervisión estructurado [15].

Este conjunto de evidencias define con precisión el problema central que motiva la presente investigación: el desarrollo de software acumula décadas de fracasos metodológicos verificables [29][30][31][32], la IA generativa tiene la capacidad técnica de transformar ese panorama [7][8][9], pero la adopción no estructurada de IA introduce riesgos que pueden agravar los problemas existentes en lugar de resolverlos [9][14][15]. La brecha no es tecnológica; es metodológica. No existe un modelo de ciclo de vida estructurado en torno al prompt como unidad de orquestación central que permita integrar la IA generativa en todas las fases del proceso de forma sistemática, evaluada, segura y gobernada. Esta investigación propone ese modelo.

1.2 Antecedentes del Problema

El panorama de fracasos documentado en la sección anterior no es un fenómeno reciente ni circunstancial. La crisis del software tiene antecedentes que se remontan a la década de 1960, cuando la NATO Software Engineering Conference de

1968 acuñó formalmente el término "crisis del software" para describir la incapacidad sistemática de la industria de entregar sistemas funcionales en tiempo y dentro del presupuesto [29]. Más de medio siglo después, los datos del CHAOS Report 2020 demuestran que esta crisis estructural no ha sido resuelta por ninguna de las metodologías desarrolladas en ese intervalo [29][33]. Es en este contexto de búsqueda histórica de soluciones donde se inscriben los antecedentes directos de la presente investigación.

La relación entre inteligencia artificial y el ciclo de vida del software tiene antecedentes que se remontan al uso de algoritmos clásicos de aprendizaje automático. La revisión de Padmanaban y Sharma documenta cómo técnicas como las redes neuronales y la lógica difusa ya eran aplicadas en fases específicas del SDLC para tareas de predicción de defectos, optimización de pruebas y análisis de requisitos [5]. Sin embargo, estos enfoques eran puntuales, no integrados, y dependían de grandes volúmenes de datos etiquetados específicos para cada tarea, lo que limitaba su generalización a equipos de desarrollo convencionales [5].

Un salto cualitativo llegó con el estudio bibliométrico de Navaei y Tabrizi, que analizó casi 150 artículos científicos en ACM, IEEE y Springer para mapear la distribución del Machine Learning (ML) a través de las fases del SDLC [6]. Sus hallazgos revelaron un desequilibrio preocupante: la fase de Pruebas concentraba la mayor cantidad de publicaciones, favorecida por la disponibilidad de datasets cuantificables, mientras que la fase de Análisis de Requisitos presentaba el menor número de estudios, limitada por la escasez de datos y la complejidad semántica de la tarea [6]. Este mapeo inicial demostró que la integración de IA en el SDLC era fragmentada, reactiva y no sistémica: cada investigación abordaba una fase de forma aislada sin proponer un modelo integral que conectara todas las etapas bajo una lógica unificada.

El panorama cambió de forma acelerada con la irrupción de los Modelos de Lenguaje de Gran Escala (*Large Language Models*, LLMs). Hou et al. publicaron la revisión sistemática de literatura más exhaustiva hasta la fecha, analizando 395 estudios y catalogando 85 tareas distintas del SDLC que pueden ser abordadas con LLMs [7]. En paralelo, Zhang et al. revisaron 1009 papers e identificaron 947 estudios que aplican 62 LLMs diferentes en 112 tareas distribuidas en las cinco grandes fases del ciclo de vida del software [8]. Estas dos revisiones, tomadas en conjunto, establecen con solidez empírica que los LLMs han alcanzado una cobertura prácticamente total del SDLC: no existe fase del ciclo de vida que no pueda ser asistida, acelerada o

parcialmente automatizada mediante el uso de modelos de lenguaje bien orquestados [7] [8].

El estudio comparativo de Abdalhamid y Almabruk cuantifica el impacto de esta integración: la incorporación de IA en el SDLC produce una aceleración del 40% en los ciclos de desarrollo y una reducción del 60% en el tiempo dedicado a la elicitación de requisitos mediante técnicas de Procesamiento de Lenguaje Natural (NLP) [9]. Asimismo, el uso de herramientas como GitHub Copilot acelera la fase de codificación en un 35% [9]. Sin embargo, el mismo estudio advierte que el código producido por IA presenta hasta un 25% más de vulnerabilidades de seguridad cuando no existe un proceso de revisión humana estructurado, y la usabilidad de los diseños generados por IA puede reducirse hasta en un 20% cuando no se aplican criterios de validación explícitos [9]. Este escenario de alto potencial y alto riesgo simultáneo define con precisión el problema central que motiva la presente investigación.

1.3 Formulación del Problema

A pesar de la evidencia empírica que demuestra la capacidad transformadora de los LLMs en el SDLC, persiste una brecha metodológica fundamental: no existe un modelo de ciclo de vida de desarrollo de software estructurado en torno al prompt como unidad de orquestación central, que permita integrar la IA generativa en todas las fases del proceso de forma sistemática, evaluada, segura y gobernada.

Esta brecha se manifiesta en múltiples niveles. En primer lugar, los equipos de desarrollo carecen de un marco que les indique cómo diseñar el prompt adecuado para cada fase del SDLC, considerando las particularidades técnicas, de seguridad y de dominio de cada etapa. El diseño del prompt no es una tarea trivial: es una disciplina de ingeniería que requiere dominar técnicas como Zero-Shot, Few-Shot, Chain of Thought (CoT), ReAct y Retrieval-Augmented Generation (RAG), cada una con aplicaciones y limitaciones específicas en el contexto del desarrollo de software

[10]. En segundo lugar, no existe un proceso de evaluación y refinamiento del prompt que sea estándar, repetible y medible. El análisis empírico de Midolo et al. sobre más de 4,5 millones de prompts reales utilizados para generación de código derivó 10 guías prácticas que maximizan la calidad del output, concluyendo que la mayoría de los desarrolladores no aplica ninguna de ellas de forma sistemática [11]. Esta ausencia de estandarización convierte el uso del prompt en una práctica artesanal e irreproducible.

En tercer lugar, la transición del uso aislado del prompt hacia la automatización agéntica —donde agentes IA ejecutan secuencias autónomas de prompts para completar fases completas del SDLC sin intervención continua del desarrollador— no cuenta con un marco de referencia consolidado y accesible para equipos convencionales [12] [13]. Hymel propone el V-Bounce model, en el que la IA actúa como motor de implementación mientras los humanos asumen el rol de validadores, logrando reducir el costo de desarrollo en 6.000 veces respecto al modelo humano-céntrico tradicional [12]. Por su parte, Raja SP describe desde Amazon Web Services el AI-DLC, un ciclo de vida AI-native con fases redefinidas (Inception, Construction, Operations) donde los agentes IA ejecutan las tareas de construcción de forma autónoma [13]. Ambos modelos demuestran que la reconfiguración del SDLC en torno al prompt y a los agentes IA es técnicamente viable, pero ninguno de los dos provee un framework metodológico completo que sea operacionalizable por equipos de desarrollo en contextos organizacionales reales.

La situación en el contexto latinoamericano agrava este diagnóstico. Paladino y Pons identifican que los principales obstáculos para integrar IA en el SDLC en la región no son tecnológicos sino metodológicos: ausencia de procesos estructurados, falta de competencias en ingeniería de prompts y desconocimiento de los riesgos de seguridad del código generado por IA [14]. En el contexto peruano específicamente, Rivas Verastegui, Tirado Ruiz y Torres Villanueva documentan que la adopción de herramientas como GitHub Copilot ya es una realidad en empresas y universidades del país, pero que los programadores novatos —que constituyen la mayoría de la

fuerza laboral en el sector— generan código significativamente menos seguro y experimentan pérdida de habilidades fundamentales cuando no existe un proceso de supervisión estructurado que oriente el uso de la IA [15].

Por todo lo expuesto, se formula el siguiente **problema de investigación**:

¿En qué medida un ciclo de vida de desarrollo de software centrado en la ingeniería del prompt —que integre el diseño, la evaluación, la aplicación por fases, la automatización agéntica y la gobernanza del prompt— mejora la eficiencia del proceso de desarrollo de software respecto a los enfoques tradicionales y a los enfoques de integración puntual de IA generativa?

1.4 Justificación de la Investigación

1.4.1 Justificación Teórica

La literatura científica existente aborda el uso de LLMs en el SDLC desde perspectivas aisladas. Los trabajos de Shrivastava et al. e Iusztin y Labonne se concentran en la ingeniería del prompt y la producción de aplicaciones con LLMs [10] [16]. Las revisiones de Fan et al., Hou et al. y Zhang et al. catalogan las tareas del SDLC abordables con LLMs, pero sin proponer un ciclo integrado [17] [7] [8]. Las propuestas de ciclos AI-native de Hymel y Raja SP ofrecen visiones sistémicas, pero carecen de operacionalización metodológica detallada [12] [13]. Los estudios de MLOps y DevOps para IA de Macha y Pulichikkunnu y de Amaro, Pereira y Miranda Silva abordan la dimensión operativa, pero tampoco integran la ingeniería del prompt como eje estructurante [18] [19]. Ningún trabajo existente unifica todas estas dimensiones en un modelo de ciclo de vida cohesionado y operacionalizable. La presente investigación llena ese vacío al proponer un marco teórico en cinco dimensiones que articula el prompt como eje central del nuevo SDLC, constituyendo una contribución original al campo de la ingeniería de software.

1.4.2 Justificación Práctica

El impacto económico de un SDLC centrado en el prompt es cuantificable y significativo. El costo de generación de código con IA es de \$0.002 por línea, frente a \$12 por línea con un desarrollador humano, lo que representa una reducción de costos de 6.000 veces [12]. GitHub Copilot incrementa la productividad en un 55,8% en tareas controladas [15], y la cobertura de pruebas automáticas supera el 70% cuando se aplica IA de forma estructurada [12]. Si un equipo de desarrollo puede alcanzar estos resultados de forma sistemática y reproducible —en lugar de por ensayo y error individual—, el impacto para empresas peruanas y latinoamericanas es inmediato. Krishna y Meda documentan casos reales de integración de IA en DevOps, Testing y SRE que demuestran reducciones sostenidas de tiempo y costo en organizaciones de distintos tamaños [20], reforzando la viabilidad práctica del enfoque propuesto.

1.4.3 Justificación Metodológica

La investigación de Li et al. representa el único caso de estudio conocido que aplica prompts de forma sistemática en todas las fases del SDLC de forma simultánea [21]. Sus resultados demuestran la viabilidad técnica del enfoque, pero identifican como limitación principal la ausencia de un framework metodológico que guíe el proceso de forma estructurada [21]. La presente investigación toma ese hallazgo como punto de partida y propone precisamente ese framework ausente. Complementariamente, el benchmark SWE-BENCH de Jimenez et al., presentado en ICLR 2024, establece métricas objetivas para evaluar la capacidad de los LLMs de resolver issues reales de GitHub, proveyendo un estándar de evaluación riguroso para medir los resultados del ciclo propuesto [22]. La disponibilidad de estas métricas hace que el modelo sea verificable y comparable con investigaciones futuras.

1.4.4 Justificación Social y Pertinencia Local

El problema no es abstracto ni ajeno al contexto peruano. La adopción de IA en programación es un fenómeno presente y creciente en el país, con impactos diferenciados según el nivel de experiencia del programador [15]. La ausencia de un

marco metodológico adecuado hace que los beneficios se concentren en los desarrolladores más experimentados, mientras que la mayoría —programadores junior— enfrenta riesgos reales de generar código inseguro, desarrollar dependencias tecnológicas sin comprensión profunda y erosionar competencias fundamentales [15]. Paladino y Pons confirman que esta asimetría se reproduce en el contexto latinoamericano en general, donde la adopción de IA en el SDLC avanza más rápido que la generación de marcos metodológicos que la orienten [14]. Un ciclo de vida estructurado en torno al prompt responsable atiende directamente esta brecha, democratizando el acceso a los beneficios de la IA generativa en el desarrollo de software.

1.5 Objetivos de la Investigación

Objetivo General

Proponer un modelo de ciclo de vida de desarrollo de software centrado en la ingeniería del prompt (Prompt-Centered SDLC) que integre de forma sistemática la inteligencia artificial generativa en cinco dimensiones —diseño e ingeniería del prompt, evaluación y refinamiento del prompt, aplicación del prompt en las fases del SDLC, automatización agéntica, y seguridad y gobernanza— con el propósito de mejorar la eficiencia, calidad y sostenibilidad del proceso de desarrollo de software.

Objetivos Específicos

OE1. Analizar el estado del arte sobre las metodologías SDLC tradicionales y AI-native, identificando sus limitaciones estructurales y las oportunidades que los LLMs ofrecen para superar dichas limitaciones [3] [4] [5] [6] [7] [8] [9].

OE2. Caracterizar los principios, técnicas y guías empíricas del diseño e ingeniería del prompt aplicados al contexto del desarrollo de software, estableciendo un corpus de buenas prácticas operacionalizables por equipos de desarrollo [10] [11] [16] [17].

OE3. Definir un proceso de evaluación, refinamiento y gestión del prompt que sea estándar, repetible y medible, utilizando métricas como pass@k, ROUGE, BLEU y benchmarks como SWE-BENCH [8] [11] [21] [22].

OE4. Documentar la aplicación del prompt como habilitador de cada fase del SDLC —requisitos, diseño, implementación, testing y mantenimiento— identificando los tipos de prompt más efectivos por fase y las posibilidades de fusión de fases gracias a prompts de alta especificidad técnica [9] [12] [13] [20] [21].

OE5. Establecer un marco de seguridad, gobernanza y monitoreo del ciclo AI-native que mitigue los riesgos identificados en la literatura —vulnerabilidades de seguridad en código generado por IA, dependencia tecnológica y pérdida de habilidades en el equipo humano— y garantice la sostenibilidad del ciclo en entornos productivos reales [15] [23] [24] [25].

1.6 Delimitación de la Investigación

La presente investigación se delimita a las cinco dimensiones del ciclo propuesto (Prompt-Centered SDLC) y su fundamentación en la literatura científica existente. No aborda el fine-tuning de modelos de lenguaje ni el entrenamiento de modelos propietarios, aspectos que si bien son relevantes en el campo de la IA, exceden el alcance de un ciclo de vida orientado a equipos de ingeniería de software convencionales. El modelo propuesto asume el uso de LLMs como servicios accesibles (GPT-4, Claude, Gemini, Llama), en línea con el enfoque de democratización documentado en la literatura reciente [10] [25]. En cuanto al alcance geográfico y de pertinencia, la investigación prioriza el contexto latinoamericano con énfasis en Perú, reconociendo que las condiciones de adopción tecnológica, los perfiles de los equipos de desarrollo y los marcos regulatorios difieren de los contextos norteamericanos y europeos donde se concentra la mayor parte de la literatura revisada [14] [15].

CAPÍTULO 2: MARCO TEÓRICO Y ESTADO DEL ARTE

2.1 Introducción al Capítulo

El presente capítulo establece el fundamento teórico y el estado del arte sobre el cual se construye el modelo Prompt-Centered SDLC propuesto en esta investigación. Se organiza en dos grandes bloques. El primero revisa los conceptos clave que articulan el marco teórico: el ciclo de vida del desarrollo de software, la inteligencia artificial generativa, la ingeniería del prompt y los modelos AI-native. El segundo presenta el estado del arte estructurado en las cinco dimensiones que conforman el modelo propuesto, identificando para cada una los aportes, limitaciones y brechas de la literatura existente que justifican la contribución original de esta investigación.

2.2 Marco Teórico

2.2.1 El Ciclo de Vida del Desarrollo de Software (SDLC)

El Software Development Life Cycle (SDLC) es el conjunto de fases, procesos y entregables que estructuran el desarrollo de un sistema de software desde su concepción hasta su retiro. La revisión comprensiva de Yas, ALazzawi y Rahmatullah establece que los modelos SDLC existentes pueden clasificarse en cuatro grandes familias: lineales (Waterfall), iterativos (Espiral), ágiles (Scrum, Kanban, XP) e híbridos, cada uno con ventajas y limitaciones específicas según el tipo de proyecto, el tamaño del equipo y los requisitos de calidad [3]. El modelo Waterfall, propuesto originalmente por Royce en 1970, establece una secuencia estrictamente lineal de fases —Requisitos, Diseño, Implementación, Verificación y Mantenimiento— que garantiza trazabilidad completa pero carece de flexibilidad ante cambios [3]. Las metodologías Agile surgieron precisamente como respuesta a esta rigidez, priorizando la entrega incremental, la colaboración continua con el cliente y la capacidad de adaptación [4].

La revisión sistemática de Aleryani, Alsabry y Alramada sobre metodologías Agile confirma que marcos como Scrum, Kanban y Extreme Programming (XP) han demostrado mejoras significativas en la satisfacción del cliente y en la reducción de tiempos de entrega respecto al modelo Waterfall [4]. Sin embargo, el mismo estudio identifica limitaciones persistentes: la gestión del conocimiento en equipos distribuidos, la dificultad de escalar Agile en proyectos grandes y la ausencia de métricas estandarizadas de calidad siguen siendo desafíos sin resolver [4]. La revisión empírica de Tetteh sobre metodologías Agile agrega evidencia cuantitativa: los equipos que aplican Scrum reportan una reducción promedio del 37% en el tiempo de entrega respecto a Waterfall, pero enfrentan un incremento del 28% en el esfuerzo de coordinación cuando el equipo supera los 10 integrantes [26].

El mantenimiento de software, frecuentemente subestimado, representa entre el 60% y el 80% del costo total del ciclo de vida de un sistema [24]. El estudio de Almashhadani, Mishra y Yazici sobre prácticas de mantenimiento con métodos Agile en entornos de nube documenta que la adopción de enfoques iterativos reduce el tiempo de resolución de defectos en producción en un 45%, pero requiere la implementación de pipelines de integración continua (CI/CD) para ser sostenible [24]. Esta evidencia refuerza la necesidad de un SDLC que no trate el mantenimiento como una fase residual, sino como una etapa integrada y continua del ciclo, especialmente en sistemas que incorporan modelos de IA generativa que evolucionan con el tiempo.

2.2.2 La Inteligencia Artificial Generativa y los LLMs

La Inteligencia Artificial Generativa (IAG) comprende los sistemas capaces de producir contenido nuevo —texto, código, imágenes, datos— a partir de patrones aprendidos durante el entrenamiento en grandes corpus de datos [10]. En el contexto

del desarrollo de software, la forma más relevante de IAG son los Modelos de Lenguaje de Gran Escala (LLMs, por sus siglas en inglés), sistemas basados en la arquitectura Transformer que han demostrado capacidades extraordinarias para comprender y generar código en múltiples lenguajes de programación [10].

Shrivastava et al. establecen la taxonomía de modelos relevantes para el desarrollo de software: los LLMs de propósito general como GPT-4 (OpenAI), Claude (Anthropic), Gemini (Google) y Llama (Meta), y los modelos especializados en código como GitHub Copilot (basado en Codex) y Amazon CodeWhisperer [10]. Iusztin y Labonne complementan esta taxonomía describiendo los patrones de ingeniería para construir aplicaciones sobre estos modelos: desde el uso directo de la API hasta arquitecturas complejas que combinan RAG, fine-tuning y sistemas multi-agente [16]. Huyen define el campo emergente del AI Engineering como la disciplina que se ocupa de construir aplicaciones útiles, seguras y sostenibles sobre Foundation Models, diferenciándolo del ML Engineering clásico por su énfasis en la adaptación de modelos pre-entrenados en lugar de su entrenamiento desde cero [25].

La capacidad de los LLMs para abarcar el SDLC completo queda documentada en dos revisiones sistemáticas de gran escala. Hou et al. identifican 85 tareas distintas del ciclo de vida del software que pueden ser abordadas con LLMs, incluyendo generación de requisitos, producción de código, detección de bugs, generación de pruebas y documentación automática [7]. Zhang et al. amplían este panorama a 112 tareas distribuidas en cinco fases del SDLC, analizadas en 947 estudios que cubren 62 LLMs diferentes, concluyendo que los modelos más recientes (GPT-4, Claude 3, Gemini Ultra) superan a los especialistas humanos en tareas específicas de codificación y pruebas cuando se les provee el contexto adecuado mediante prompts bien diseñados [8].

2.2.3 La Ingeniería del Prompt

El prompt es la instrucción o conjunto de instrucciones que un usuario o sistema proporciona a un LLM para obtener un output deseado. Lejos de ser una simple cadena de texto, el diseño del prompt es una disciplina de ingeniería con técnicas formalizadas, métricas de evaluación y ciclos de mejora iterativa [10] [11].

Shrivastava et al. describen la anatomía de un prompt efectivo como la combinación de cuatro elementos: contexto (información de fondo que orienta al modelo), instrucción (la tarea específica a realizar), ejemplos (cuando se aplica Few-Shot prompting) y restricciones (límites de formato, longitud o contenido del output) [10]. Las técnicas principales de prompting documentadas en la literatura son: Zero-Shot (instrucción directa sin ejemplos), Few-Shot (instrucción con ejemplos representativos), Chain of Thought o CoT (instrucción que induce razonamiento paso a paso), ReAct (combinación de razonamiento y acción para tareas agénticas) y Retrieval-Augmented Generation o RAG (incorporación de contexto externo recuperado dinámicamente) [10] [16].

El estudio empírico de Midolo et al., que analizó más de 4,5 millones de prompts reales utilizados para generación de código, es la investigación más específica disponible sobre la efectividad del prompt en tareas de desarrollo de software [11]. Sus 10 guías derivadas establecen que la calidad del output de código mejora significativamente cuando el prompt: especifica el lenguaje de programación y la versión, incluye el contexto del sistema existente, define explícitamente los criterios de aceptación, proporciona ejemplos de entrada y salida esperada, e indica restricciones de seguridad y compatibilidad [11]. La investigación de Jiang et al. complementa estos hallazgos desde una perspectiva de fine-tuning: el Comparative Prefix-Tuning, técnica que entrena al modelo con pares de código de alta y baja calidad como guía comparativa, logra mejorar la calidad del código generado en métricas de correctitud funcional (pass@k) y legibilidad respecto al prompting sin ajuste [27].

2.2.4 Los Modelos AI-Native del SDLC

Los modelos AI-native del SDLC son propuestas que no se limitan a incorporar herramientas de IA en el ciclo tradicional, sino que reimaginan el proceso completo bajo la premisa de que la IA es el actor principal de la ejecución y el humano es el actor principal de la validación.

El V-Bounce model propuesto por Hymel adapta el clásico V-model de verificación y validación introduciendo un “rebote” (bounce) en la base del diagrama: la IA genera el código de forma casi instantánea y a costo mínimo, mientras que los humanos desplazan su esfuerzo hacia los extremos del ciclo —definición de requisitos y arquitectura a la izquierda, validación y aprobación a la derecha [12]. Este modelo incorpora seis pasos por fase: Input, AI Generation, Human Review, Refinement, Approval y Knowledge Capture, garantizando que el conocimiento organizacional quede sistematizado en cada iteración [12]. El AI-DLC definido por Raja SP desde Amazon Web Services propone tres macrofases redefinidas: Inception (planificación y arquitectura AI-first), Construction (generación y validación automatizada) y Operations (monitoreo y mejora continua con IA), articuladas por un sistema de agentes IA que ejecutan las tareas de construcción de forma autónoma y reportan al equipo humano para su aprobación [13].

Ambos modelos coinciden en un principio central: la productividad no mejora añadiendo IA a un proceso humano-céntrico, sino rediseñando el proceso bajo la lógica de que la IA puede ejecutar la mayoría de las tareas repetitivas y técnicas del SDLC cuando recibe instrucciones precisas y bien estructuradas. El prompt es, en este marco, la interfaz principal entre el equipo humano y el sistema de IA [12] [13].

2.3 Estado del Arte por Dimensiones

El estado del arte se presenta organizado en las cinco dimensiones del modelo Prompt-Centered SDLC propuesto. Para cada dimensión se describen los aportes de

la literatura, las limitaciones identificadas y la brecha específica que justifica la contribución de esta investigación.

2.3.1 Dimensión 1: Diseño e Ingeniería del Prompt para el SDLC

Aportes de la literatura. La investigación sobre ingeniería del prompt para tareas de desarrollo de software ha avanzado de forma notable en los últimos tres años. Fan et al. identifican el prompt engineering dinámico —la capacidad de adaptar el prompt en tiempo de ejecución según el contexto de la tarea— como uno de los ocho problemas abiertos más relevantes del campo, señalando que la mayoría de los sistemas actuales usan prompts estáticos que no capturan la variabilidad de los problemas reales de ingeniería de software [17]. Hou et al. documentan que las técnicas de optimización del prompt, incluyendo Few-Shot prompting y RAG, son las estrategias más frecuentemente citadas para mejorar el desempeño de los LLMs en las 85 tareas del SDLC que analizan [7]. Zhang et al. refuerzan este hallazgo, reportando que el uso de CoT mejora hasta en un 34% la correctitud del código generado en tareas de razonamiento lógico complejo [8].

El aporte más específico al diseño del prompt para código proviene de Midolo et al., cuyas 10 guías empíricas representan el estado del arte más actualizado y con mayor base empírica disponible para esta dimensión [11]. Su análisis de 4,5 millones de prompts reales permite afirmar, por primera vez con evidencia estadística robusta, cuáles características del prompt tienen mayor impacto en la calidad del código generado [11]. Li et al. amplían este conocimiento con su caso de estudio de aplicación de prompts en el SDLC completo, demostrando que los prompts diseñados con contexto de sistema, especificación de requisitos y ejemplos de código similar producen outputs de mayor calidad y menor necesidad de refinamiento [21].

Limitaciones identificadas. A pesar de estos avances, la literatura sobre diseño del prompt para el SDLC presenta tres limitaciones importantes. Primera, la mayoría de los estudios aborda el prompt para tareas aisladas (generación de código, detección de bugs, generación de pruebas) sin proponer una taxonomía integrada de tipos de prompt por fase del ciclo de vida [7] [8]. Segunda, las guías y técnicas existentes no han sido validadas en contextos de desarrollo reales con equipos multidisciplinares, sino principalmente en benchmarks académicos [11] [21]. Tercera, no existe un corpus sistematizado de prompts exitosos por fase del SDLC que pueda ser reutilizado y adaptado por equipos de desarrollo, lo que obliga a cada equipo a redescubrir por ensayo-error los patrones que funcionan [17].

Brecha identificada. No existe una guía de diseño de prompts estructurada por fase del SDLC que integre las técnicas documentadas en la literatura con las particularidades técnicas, de seguridad y de dominio de cada etapa del ciclo de vida. Esta brecha es la que la Dimensión 1 del modelo propuesto busca cubrir.

2.3.2 Dimensión 2: Evaluación, Refinamiento y Gestión del Prompt

Aportes de la literatura. La evaluación de los outputs generados por LLMs en tareas de software ha sido abordada desde múltiples perspectivas métricas. Zhang et al. documentan el uso de métricas como BLEU (*Bilingual Evaluation Understudy*), ROUGE (*Recall-Oriented Understudy for Gisting Evaluation*) y CodeBLEU para evaluar la similitud entre código generado y código de referencia, señalando que estas métricas capturan la corrección sintáctica pero no siempre la corrección funcional [8]. El benchmark SWE-BENCH de Jimenez et al. representa el avance más significativo en evaluación funcional real: propone resolver issues auténticos de repositorios GitHub populares y evalúa el éxito mediante la ejecución de los test suites del proyecto, midiendo la tasa de resolución completa (pass@1) como métrica principal [22]. Este enfoque supera las limitaciones de las métricas basadas en similitud textual al evaluar si el código generado realmente funciona en el contexto del sistema existente [22].

El refinamiento iterativo del prompt ha sido estudiado por Midolo et al., quienes documentan que la aplicación secuencial de sus 10 guías en múltiples iteraciones produce mejoras acumulativas en la calidad del código, con rendimientos decrecientes a partir de la tercera iteración [11]. Jiang et al. proponen el Comparative Prefix-Tuning como una técnica de refinamiento a nivel de modelo: al ajustar el LLM con pares de código bueno/malo como señal de calidad, el modelo internaliza los criterios de calidad y requiere prompts menos elaborados para producir outputs de alta calidad [27]. Iusztin y Labonne abordan la gestión del prompt desde una perspectiva de ingeniería de sistemas, proponiendo el versionado de prompts como práctica esencial para garantizar la reproducibilidad de los resultados y la trazabilidad de los cambios en sistemas de producción [16].

Limitaciones identificadas. La principal limitación de la literatura en esta dimensión es la ausencia de un proceso unificado de evaluación-refinamiento-gestión que sea aplicable como ciclo repetible en el contexto del SDLC. Las métricas de evaluación (Zhang et al. [8], Jimenez et al. [22]), las técnicas de refinamiento (Midolo et al. [11], Jiang et al. [27]) y las prácticas de gestión y versionado (Iusztin y Labonne [16]) han sido desarrolladas de forma independiente, sin un marco que las integre en un flujo de trabajo coherente para equipos de desarrollo.

Brecha identificada. No existe un proceso estándar, repetible y medible que combine la evaluación del output, el refinamiento iterativo del prompt y el versionado del prompt como práctica de ingeniería aplicable en el contexto del SDLC. La Dimensión 2 del modelo propuesto cubre esta brecha.

2.3.3 Dimensión 3: El Prompt como Habilitador de las Fases del SDLC

Aportes de la literatura. Esta es la dimensión con mayor respaldo empírico en la literatura. La evolución histórica documentada por Padmanaban y Sharma establece

que el uso de IA en el SDLC no es nuevo: desde 2010, técnicas de aprendizaje automático clásico eran aplicadas en fases específicas del ciclo, aunque de forma fragmentada y dependiente de datos etiquetados [5]. La revisión de Navaei y Tabrizi amplía este panorama histórico, demostrando que para el período 2015-2021 la investigación sobre ML en el SDLC ya cubría todas las fases, aunque de forma desigual [6].

Con la llegada de los LLMs, la cobertura se volvió prácticamente total y cualitativamente diferente. En la fase de Análisis de Requisitos, Abdalhamid y Almabruk documentan que el uso de NLP y LLMs reduce el tiempo de elicitación en un 60% y mejora la completitud de los requisitos al detectar inconsistencias y ambigüedades que los métodos manuales frecuentemente omiten [9]. En la fase de Diseño, los mismos autores reportan que las herramientas generativas permiten producir arquitecturas candidatas y diagramas UML en minutos, aunque señalan que la usabilidad de los diseños generados puede reducirse hasta en un 20% cuando no se aplican criterios de validación humana explícitos [9]. En la fase de Implementación, Rivas Verastegui, Tirado Ruiz y Torres Villanueva documentan que GitHub Copilot incrementa la productividad de codificación en un 55,8% en experimentos controlados [15], mientras que Abdalhamid y Almabruk reportan una aceleración del 35% en tareas de codificación rutinaria [9].

El caso de estudio de Li et al. es el aporte más integral disponible para esta dimensión: aplicaron prompts especializados en todas las fases del SDLC de un proyecto real, demostrando que es técnicamente posible orquestar un ciclo completo de desarrollo mediante una secuencia estructurada de prompts [21]. El V-Bounce model de Hymel y el AI-DLC de Raja SP proveen la visión arquitectural de cómo este ciclo puede institucionalizarse: el primero mediante la redefinición del V-model clásico [12], y el segundo mediante la adopción de fases AI-native con agentes como actores de ejecución [13].

La evidencia sobre metodologías Agile es igualmente relevante para esta dimensión. Aleryani, Alsabry y Alramada documentan que Scrum y Kanban son los marcos más adoptados y que su combinación con herramientas de IA generativa está emergiendo como una práctica común en equipos de alto desempeño [4]. Tetteh refuerza esta tendencia con datos empíricos que muestran que los equipos Agile que incorporan asistentes de código IA reportan una reducción del 42% en el tiempo de resolución de bugs y un 38% de reducción en el tiempo dedicado a la documentación [26]. Almashhadani, Mishra y Yazici añaden la perspectiva del mantenimiento en entornos de nube, documentando que las prácticas Agile combinadas con CI/CD reducen el tiempo de resolución de defectos en producción y facilitan la actualización continua de los modelos de IA incorporados al sistema [24].

Limitaciones identificadas. A pesar de la abundante evidencia, la literatura no ofrece un mapa sistemático de qué tipo de prompt es más efectivo para cada fase específica del SDLC, considerando las particularidades técnicas de cada etapa. La mayoría de los estudios evalúa el impacto de la IA en fases aisladas, sin proponer

una articulación entre fases que aproveche la continuidad del contexto acumulado [7] [8] [21].

Brecha identificada. No existe una guía operacional que especifique, para cada fase del SDLC, el tipo de prompt más efectivo, las métricas de calidad aplicables y las condiciones bajo las cuales dos fases pueden fusionarse (por ejemplo, implementación y testing simultáneos) gracias a un prompt de alta especificidad técnica que incorpore criterios de ambas fases. La Dimensión 3 del modelo propuesto aborda esta brecha.

2.3.4 Dimensión 4: Automatización Agéntica y Pipelines IA en el SDLC

Aportes de la literatura. La automatización agéntica representa la evolución natural del uso del prompt: en lugar de interacciones aisladas usuario-LLM, los agentes IA ejecutan secuencias autónomas de prompts, utilizan herramientas externas (compiladores, intérpretes, APIs, bases de datos) y toman decisiones intermedias para completar tareas complejas del SDLC sin intervención continua del desarrollador [10] [12] [13]. Fan et al. identifican los LLMs como agentes autónomos capaces de planificar, ejecutar y verificar tareas de ingeniería de software como uno de los temas emergentes más relevantes del campo, señalando casos como MetaGPT —sistema multi-agente que simula un equipo completo de desarrollo de software— como evidencia del potencial de este enfoque [17].

La dimensión operativa de la automatización agéntica —su integración con pipelines de DevOps— ha sido estudiada por Macha y Pulichikkunnu, quienes documentan la evolución de DevOps hacia MLOps y LLMOps: conjuntos de prácticas para gestionar el ciclo de vida de modelos de IA en producción, incluyendo versionado de datos, registro de modelos (model registry), monitoreo de drift, entrenamiento continuo (CT) y despliegue automatizado mediante pipelines CI/CD/CT [18]. Amaro, Pereira y Miranda Silva complementan este análisis con su revisión sistemática que mapea las capacidades DevOps a las fases del SDLC, identificando 47 prácticas DevOps que tienen correspondencia directa con alguna fase del ciclo de vida del software y que pueden ser automatizadas mediante agentes IA [28].

Iusztin y Labonne proveen la perspectiva de ingeniería de sistemas para esta dimensión: su descripción de los pipelines de producción de aplicaciones LLM —desde la ingesta de datos hasta el monitoreo en producción— establece los componentes técnicos necesarios para construir un SDLC AI-native sostenible [16]. Huyen refuerza esta perspectiva desde el AI Engineering, documentando patrones de arquitectura para aplicaciones con Foundation Models que incluyen caché de prompts, gestión de contexto y sistemas de retroalimentación que permiten la mejora continua del sistema [25].

Limitaciones identificadas. La principal brecha en esta dimensión es la ausencia de un framework que integre la automatización agéntica del SDLC con las prácticas de

ingeniería del prompt. Los estudios de agentes IA (Fan et al. [17], Hymel [12], Raja SP [13]) y los estudios de MLOps/DevOps (Macha y Pulichikkunnu [18], Amaro et al. [28]) han evolucionado en paralelo sin una articulación explícita que defina cómo el diseño del prompt orquesta los agentes y cómo los pipelines MLOps garantizan la sostenibilidad del ciclo agéntico.

Brecha identificada. No existe un modelo que integre los principios de ingeniería del prompt, la arquitectura de agentes IA y las prácticas MLOps/ DevOps en un framework unificado de automatización agéntica del SDLC. La Dimensión 4 del modelo propuesto cubre esta brecha.

2.3.5 Dimensión 5: Seguridad, Gobernanza y Monitoreo del Ciclo AI-Native

Aportes de la literatura. La seguridad del código generado por IA es uno de los temas de investigación de más rápido crecimiento en el campo. El estudio comprensivo de Dai, Xu y Tao sobre la generación segura de código con LLMs documenta que los modelos actuales presentan vulnerabilidades sistemáticas en categorías como inyección SQL, buffer overflow, uso inseguro de criptografía y exposición de secretos en el código [23]. Su análisis de múltiples LLMs bajo condiciones controladas establece que ningún modelo genera código libre de vulnerabilidades de forma consistente, y que la presencia de guardrails explícitos en el prompt —instrucciones de seguridad incorporadas en el diseño del prompt mismo— reduce significativamente la frecuencia de vulnerabilidades [23]. Abdalhamid y Almabruk corroboran este hallazgo desde una perspectiva comparativa: el código generado por IA sin revisión humana presenta un 25% más de vulnerabilidades de seguridad que el código escrito por desarrolladores humanos con experiencia [9].

La gobernanza institucional del SDLC en entornos que incorporan IA ha sido documentada por la Universidad de South Dakota, cuya política oficial establece estándares de seguridad obligatorios (OWASP, CISA) que deben aplicarse desde la fase de Diseño —seguridad by design— y mantenerse en Testing (DAST, Static Code Analysis, pruebas de penetración) y Mantenimiento (patch management inmediato para vulnerabilidades críticas) [2]. Este modelo institucional demuestra que la gobernanza del SDLC AI-native no es solo una cuestión técnica, sino también organizacional y normativa.

Shrivastava et al. abordan la dimensión ética de la gobernanza, documentando los riesgos de sesgo en el código generado por IA, las técnicas de prompt injection y prompt leaking como vectores de ataque específicos de los sistemas LLM, y la necesidad de guardrails —tanto en el diseño del prompt como en la arquitectura del sistema— para garantizar el uso responsable de la IA generativa en desarrollo de software [10]. Rivas Verastegui, Tirado Ruiz y Torres Villanueva añaden la dimensión social de la gobernanza: el riesgo de dependencia tecnológica y pérdida de habilidades en programadores junior, que constituyen la mayoría de la fuerza laboral

en el contexto peruano, requiere marcos de supervisión y formación continua que el mercado actual no provee de forma sistemática [15].

Macha y Pulichikkunnu documentan la dimensión operativa de la gobernanza a través del monitoreo continuo: el drift de datos y el degradado de desempeño de los modelos en producción son fenómenos inevitables que requieren pipelines de monitoreo, alertas automáticas y procesos de reentrenamiento o actualización de prompts para garantizar la calidad sostenida del ciclo AI-native [18]. Huyen establece el marco teórico para este monitoreo desde el AI Engineering, describiendo las métricas de observabilidad específicas para sistemas con Foundation Models: latencia de inferencia, tasa de alucinaciones, calidad del retrieval en sistemas RAG y satisfacción del usuario con los outputs generados [25]. Almashhadani, Mishra y Yazici complementan esta perspectiva desde el mantenimiento en la nube, documentando que los sistemas de IA generativa requieren estrategias de mantenimiento adaptativas que combinen monitoreo automatizado con revisión humana periódica [24].

Limitaciones identificadas. La literatura sobre seguridad, gobernanza y monitoreo del SDLC AI-native está fragmentada en tres comunidades que se comunican poco entre sí: la comunidad de seguridad de IA (Dai et al. [23]), la comunidad de MLOps (Macha y Pulichikkunnu [18], Huyen [25]) y la comunidad de ingeniería de software (Abdalhamid y Almaghrabi [9], Rivas Verastegui et al. [15]). No existe un framework que integre las tres perspectivas —técnica, operativa y social— en un modelo de gobernanza aplicable a equipos de desarrollo convencionales.

Brecha identificada. No existe un marco de seguridad, gobernanza y monitoreo diseñado específicamente para el SDLC AI-native, que integre la seguridad by design en el prompt, los guardrails éticos, el monitoreo MLOps y la gestión del capital humano en un modelo operacionalizable. La Dimensión 5 del modelo propuesto cubre esta brecha.

2.4 Síntesis del Estado del Arte: Brechas que Justifican el Modelo Propuesto

La revisión del estado del arte en las cinco dimensiones permite identificar una brecha central y consistente: existe una abundante evidencia empírica sobre el potencial de la IA generativa para transformar cada fase del SDLC, pero no existe un modelo integrado que articule el prompt como eje organizador de todo el ciclo, desde su diseño hasta su gobernanza. El Prompt-Centered SDLC propuesto en esta investigación es la respuesta metodológica a esa brecha.

La siguiente tabla sintetiza los aportes, limitaciones y brechas identificadas en el estado del arte para cada dimensión del modelo propuesto:

Dimensión	Aportes principales	Limitación central	Brecha cubierta
-----------	---------------------	--------------------	-----------------

Dimensión	Aportes principales	Limitación central	Brecha cubierta
D1 – Diseño e Ingeniería del Prompt	Midolo et al. [11]; Hou et al. [7]; Zhang et al. [8]; Fan et al. [17]	Sin taxonomía de prompts por fase del SDLC	Guía de diseño de prompts por fase
D2 – Evaluación, Refinamiento y Gestión	SWE-BENCH [22]; Jiang et al. [27]; Iusztin & Labonne [16]	Sin proceso unificado e integrado	Ciclo estándar de evaluación-refinamiento-gestión
D3 – Prompt en las Fases del SDLC	Li et al. [21]; Abdalhamid & Almabruk [9]; Hymel [12]; Raja SP [13]	Sin mapa de tipos de prompt por fase	Guía operacional por fase con posibilidad de fusión
D4 – Automatización Agéntica	Fan et al. [17]; Macha & Pulichikkunnu [18]; Amaro et al. [28]	Sin integración prompt + agentes + MLOps	Framework unificado de automatización agéntica
D5 – Seguridad, Gobernanza y Monitoreo	Dai et al. [23]; Shrivastava et al. [10]; Huyen [25]; Rivas et al. [15]	Sin marco integrado técnico-operativo-social	Modelo de gobernanza AI-native operacionalizable

CAPÍTULO 3: METODOLOGÍA DE LA INVESTIGACIÓN

3.1 Enfoque de la Investigación

La presente investigación adopta un **enfoque mixto**, que combina componentes cualitativos y cuantitativos de forma complementaria y secuencial. El componente cualitativo predomina en las fases de análisis del estado del arte, identificación de brechas y construcción del modelo teórico Prompt-Centered SDLC, dado que el objeto de estudio —la integración sistemática de la IA generativa en el SDLC mediante el prompt como eje central— requiere interpretación profunda de la literatura científica y síntesis conceptual de múltiples corrientes de investigación [7] [8] [10]. El componente cuantitativo se incorpora en la fase de validación, mediante el uso de métricas objetivas como pass@k , BLEU, ROUGE y los benchmarks establecidos por SWE-BENCH [22], que permiten evaluar de forma medible el desempeño del modelo propuesto en tareas concretas del SDLC.

Esta combinación se justifica por la naturaleza del problema investigado. Por un lado, la construcción del modelo Prompt-Centered SDLC requiere un análisis interpretativo y crítico de una literatura heterogénea que abarca ingeniería de software, ciencias de

la computación, gestión de proyectos y ética de la IA [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13]. Por otro lado, la propuesta solo tendrá valor académico y práctico si sus afirmaciones sobre mejora de eficiencia, calidad y sostenibilidad del proceso de desarrollo son verificables mediante indicadores cuantificables [9] [15] [22].

3.2 Tipo y Nivel de la Investigación

3.2.1 Tipo de Investigación

La investigación es de tipo **aplicada**, dado que su propósito no es generar conocimiento teórico abstracto, sino producir un modelo metodológico —el Prompt-Centered SDLC— directamente utilizable por equipos de desarrollo de software para mejorar su proceso de trabajo. Este tipo de investigación parte del conocimiento científico existente y lo transforma en una propuesta práctica orientada a resolver un problema concreto identificado en el campo [10] [12] [13]. La naturaleza aplicada de la investigación se fundamenta en la brecha identificada en el estado del arte: existe suficiente evidencia empírica sobre el potencial de la IA generativa en el SDLC, pero no existe un modelo integrado que operacionalice ese potencial de forma sistemática y accesible [7] [8] [21].

3.2.2 Nivel de la Investigación

El nivel de la investigación es **propositivo-descriptivo**. Es descriptivo en la medida en que caracteriza con detalle el estado actual de la integración de la IA generativa en el SDLC a través del análisis sistemático de la literatura existente [6] [7] [8] [9]. Es propositivo en la medida en que, a partir de ese diagnóstico descriptivo, formula un modelo original —el Prompt-Centered SDLC— que responde a las brechas identificadas y propone una nueva forma de organizar el ciclo de vida del desarrollo de software [10] [12] [13] [21].

3.3 Diseño de la Investigación

El diseño de la investigación es **no experimental, transversal y documental**. No experimental porque no se manipulan variables mediante intervención directa en el campo, sino que se trabaja con la literatura científica existente como fuente primaria de datos [5] [6] [7] [8]. Transversal porque el análisis del estado del arte se realiza en un corte temporal definido, abarcando publicaciones desde 2016 hasta 2026, con énfasis en los trabajos más recientes que reflejan el estado actual del campo [9] [10] [11] [12] [13]. Documental porque el instrumento principal de recolección de datos es la revisión sistemática y crítica de fuentes bibliográficas, políticas institucionales, white papers técnicos y libros especializados [1] [2] [3] [4] [16] [20] [25].

3.3.1 Cómo el Análisis del Estado del Arte Dio Origen al Modelo Propuesto

El proceso de construcción del modelo Prompt-Centered SDLC siguió una lógica inductiva-abductiva: a partir del análisis detallado de 26 fuentes primarias identificadas en el estado del arte, se detectaron patrones recurrentes, convergencias teóricas y brechas sistemáticas que apuntaban hacia una misma dirección: la necesidad de un ciclo de vida que articulara el prompt como unidad de orquestación central del SDLC.

Este proceso se desarrolló en cuatro etapas:

Primera etapa — Mapeo exhaustivo de la literatura. Se identificaron y analizaron 26 fuentes primarias organizadas en tres grupos: artículos y conferencias con formato de plantilla del método científico (7 fuentes), estados del arte con formato alternativo (15 fuentes) y libros especializados (4 fuentes). Cada fuente fue analizada en su totalidad, extrayendo sus aportes, limitaciones y la brecha específica que señalaba como no resuelta [1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] [20] [21] [22] [23] [24] [25] [26] [27] [28].

Segunda etapa — Identificación de convergencias. El análisis comparativo de las fuentes reveló cinco convergencias centrales: (a) todas las fases del SDLC son susceptibles de ser asistidas por LLMs mediante prompts [7] [8]; (b) la calidad del prompt determina directamente la calidad del output [11] [21]; (c) la transición hacia agentes IA autónomos es la evolución natural del uso individual del prompt [12] [13] [17]; (d) la seguridad y la gobernanza del código generado por IA son condición necesaria para la adopción sostenible del ciclo AI-native [9] [23] [10]; (e) no existe ningún modelo que integre estas cuatro convergencias en un framework unificado [7] [8] [21].

Tercera etapa — Definición de las dimensiones del modelo. A partir de las cinco convergencias identificadas, se definieron las cinco dimensiones del Prompt-Centered SDLC: Diseño e Ingeniería del Prompt (D1), Evaluación, Refinamiento y Gestión del Prompt (D2), El Prompt como Habilitador de las Fases del SDLC (D3), Automatización Agéntica y Pipelines IA (D4), y Seguridad, Gobernanza y Monitoreo (D5). Cada dimensión fue asignada con un mínimo de cuatro fuentes de respaldo, garantizando la solidez del fundamento empírico de cada componente del modelo [10] [11] [12] [13] [16] [17] [18] [21] [22] [23] [25] [27] [28].

Cuarta etapa — Validación conceptual del modelo. El modelo resultante fue contrastado con los dos marcos AI-native existentes más desarrollados en la literatura —el V-Bounce model de Hymel [12] y el AI-DLC de Raja SP [13]— verificando que el Prompt-Centered SDLC los subsume, es decir, que sus principios son coherentes con ambos marcos y los amplía al proveer la dimensión metodológica de ingeniería del prompt que ambos asumen pero no desarrollan.

3.4 Población y Unidad de Análisis

Dado el carácter documental de la investigación, la **unidad de análisis** no está compuesta por personas o empresas, sino por **fuentes bibliográficas primarias** relacionadas con la integración de la IA generativa en el SDLC.

La **población** de fuentes está delimitada por los siguientes criterios:

- **Criterio temático:** publicaciones que aborden explícitamente la relación entre IA generativa (LLMs, modelos de lenguaje, herramientas de generación de código) y alguna fase del SDLC, o que definan metodologías para el ciclo de vida del desarrollo de software en contextos que incorporan IA.
- **Criterio temporal:** publicaciones comprendidas entre 2016 y 2026, priorizando los trabajos del período 2022-2026 por su mayor relevancia para el estado actual del campo. El límite inferior de 2016 se establece para incluir las guías institucionales del SDLC clásico [1] [2] que sirven de punto de comparación con los modelos AI-native.
- **Criterio de calidad:** publicaciones en revistas indexadas (ACM, IEEE, Springer, Elsevier), conferencias de primer nivel (ICLR, ENASE, JAIIO), preprints con revisión comunitaria activa (arXiv cs.SE, cs.CL, cs.CR), white papers técnicos de organizaciones de referencia (AWS, Crowdbotics) y libros de editoriales especializadas (O'Reilly, Packt, Apress).
- **Criterio de idioma:** publicaciones en inglés y español, incluyendo explícitamente fuentes en español del contexto latinoamericano y peruano [14] [15] para garantizar la pertinencia local de la investigación.

La **muestra** de la investigación está compuesta por las **26 fuentes primarias** identificadas y analizadas en el estado del arte del Capítulo 2, más el libro base de Shrivastava et al. [10], que constituye la obra central sobre la que se construye y contrasta el modelo propuesto.

3.5 Técnicas e Instrumentos de Recolección de Datos

3.5.1 Revisión Sistemática de Literatura

La técnica principal de recolección de datos es la **revisión sistemática de literatura**, que permite identificar, seleccionar, analizar y sintetizar de forma rigurosa y reproducible el conocimiento existente sobre un tema [6] [7] [8]. A diferencia de la revisión narrativa tradicional, la revisión sistemática sigue un protocolo explícito que minimiza el sesgo de selección y garantiza la trazabilidad del proceso. En esta investigación, el protocolo de revisión sistemática siguió los principios metodológicos documentados por Hou et al. y Zhang et al. en sus propias revisiones sistemáticas de

literatura sobre LLMs en ingeniería de software [7] [8], adaptados al alcance y los recursos de la presente investigación.

Las fuentes fueron buscadas en las siguientes bases de datos: Google Scholar, ACM Digital Library, IEEE Xplore, arXiv (categorías cs.SE, cs.CL, cs.CR) y ResearchGate. Las palabras clave utilizadas incluyeron: *Generative AI in SDLC*, *Large Language Models software development*, *prompt engineering software development lifecycle*, *AI-native SDLC*, *LLMs code generation*, *MLOps AI software*, *DevOps machine learning workflows* y *AI code security*.

3.5.2 Análisis Documental

El **análisis documental** se aplicó a cada una de las 26 fuentes primarias seleccionadas, extrayendo de forma sistemática los siguientes elementos: (a) problema central abordado por el autor, (b) metodología empleada, (c) aportes principales y métricas reportadas, (d) limitaciones explícitamente reconocidas, (e) brechas señaladas como líneas futuras de investigación, y (f) variables, dimensiones e indicadores operacionalizados. Este instrumento de análisis fue diseñado para garantizar la comparabilidad entre fuentes de naturaleza heterogénea (artículos empíricos, white papers, libros, políticas institucionales).

3.5.3 Matriz de Operabilidad de Variables

El instrumento de síntesis principal de la investigación es la **Matriz de Operabilidad de Variables**, que consolida para cada dimensión del modelo Prompt-Centered SDLC: la variable conceptual, su definición operacional, los indicadores medibles, las métricas de evaluación y las fuentes bibliográficas de respaldo. Esta matriz sigue el formato estándar de investigación aplicada en ingeniería de software [1] [2] [9] [10] y constituye el puente metodológico entre el marco teórico del Capítulo 2 y la propuesta del modelo en el Capítulo 4.

3.6 Variables de la Investigación

3.6.1 Variable Independiente

Ingeniería del Prompt en el SDLC: conjunto de técnicas, procesos y prácticas de diseño, evaluación, refinamiento, versionado y aplicación del prompt como mecanismo de orquestación de la IA generativa en el ciclo de vida del desarrollo de software [10] [11] [16].

3.6.2 Variable Dependiente

Eficiencia del proceso de desarrollo de software: medida multidimensional que combina indicadores de velocidad (reducción de tiempo por fase), calidad (tasa de defectos, cobertura de pruebas, vulnerabilidades de seguridad) y sostenibilidad

(capacidad del ciclo de mantenerse en el tiempo con calidad consistente) [9] [15] [22] [25].

3.7 Matriz de Operabilidad de Variables

Variable	Definición Conceptual	Definición Operacional	Dimensión	Indicadores	Fuente
Diseño del Prompt	Proceso de construcción de instrucciones efectivas para LLMs en tareas del SDLC	Aplicación de técnicas documentadas (Zero-Shot, Few-Shot, CoT, RAG, ReAct) con anatomía completa del prompt	D1	Calidad del output (pass@k), reducción de iteraciones necesarias	[10] [11] [17]
Evaluación y Refinamiento	Proceso iterativo de medición y mejora de la calidad del prompt	Aplicación de métricas BLEU, ROUGE, pass@k y benchmarks SWE-BENCH en ciclos de mejora	D2	pass@k, BLEU, ROUGE, tasa de resolución SWE-BENCH	[8] [22] [27]
Aplicación por Fase del SDLC	Uso de prompts especializados en cada etapa del ciclo de vida	Porcentaje de automatización por fase, reducción de tiempo y errores por fase	D3	% reducción tiempo por fase, % reducción defectos, cobertura pruebas	[9] [12] [13] [21]
Automatización Agéntica	Uso de agentes IA que ejecutan secuencias de prompts de forma	Número de fases automatizadas sin intervención continua, tasa de	D4	% tareas completadas autónomamente, tiempo ciclo completo,	[12] [13] [18] [28]

	autónoma	éxito de agentes		costo por línea	
Seguridad y Gobernanza	Marco de prácticas para garantizar la seguridad, ética y monitoreo del ciclo AI-native	Frecuencia de vulnerabilidades en código generado, cobertura de guardrails, métricas de drift	D5	N.º vulnerabilidades por KLOC, cobertura OWASP, tasa de drift detectado	[2] [9] [23] [25]

3.8 Procedimiento de la Investigación

El procedimiento de la investigación se desarrolló en cinco etapas secuenciales:

Etap 1 — Revisión y análisis del estado del arte (Capítulo 2). Identificación, selección y análisis sistemático de las 26 fuentes primarias, aplicando el protocolo de revisión documental descrito en la sección 3.5. Resultado: mapa completo de aportes, limitaciones y brechas por dimensión [6] [7] [8].

Etap 2 — Identificación de convergencias y brechas. Análisis comparativo cruzado de las 26 fuentes para identificar los patrones recurrentes y las brechas sistémicas que justifican el modelo propuesto [7] [8] [10] [11] [12] [13] [21]. Resultado: cinco convergencias centrales y cinco brechas que definen las dimensiones del modelo.

Etap 3 — Construcción del modelo Prompt-Centered SDLC (Capítulo 4). Definición de las cinco dimensiones del modelo, con su fundamento bibliográfico, sus principios operativos y sus indicadores de evaluación [10] [11] [12] [13] [16] [17] [18] [21] [22] [23] [25] [27] [28]. Resultado: modelo teórico completo con matriz de operabilidad de variables.

Etap 4 — Validación conceptual del modelo. Contraste del modelo propuesto con los marcos AI-native existentes (V-Bounce [12], AI-DLC [13]) y con los estudios de aplicación integral del prompt en el SDLC (Li et al. [21]). Resultado: verificación de coherencia y complementariedad del modelo propuesto respecto al estado del arte.

Etap 5 — Consolidación y redacción final. Integración de todos los componentes en el documento final de la investigación, asegurando la consistencia interna entre el marco teórico, la metodología y el modelo propuesto. Resultado: plan de tesis completo y fundamentado.

CAPÍTULO 4: GESTIÓN ADMINISTRATIVA

4.1 Introducción al Capítulo

El presente capítulo consolida los aspectos administrativos que sustentan la ejecución del plan de investigación. Se organiza en tres secciones: el cronograma de actividades, el presupuesto estimado y la fuente de financiamiento. El cronograma distingue dos fases: la fase de elaboración del Plan de Tesis (marzo–mayo 2026) y la fase de desarrollo de la Tesis Completa (mayo–junio 2026), que incluye los capítulos de Resultados de la Investigación, Intervención Metodológica y Anexos.

4.2 Cronograma de Actividades

4.2.1 Fase I — Elaboración del Plan de Tesis

Período: 20 de marzo de 2026 – 8 de mayo de 2026

N.º	Actividad	Sem 1 (20–26 mar)	Sem 2 (27 mar– 2 abr)	Sem 3 (3–9 abr)	Sem 4 (10–16 abr)	Sem 5 (17–23 abr)	Sem 6 (24–30 abr)	Sem 7 (1–7 may)	Sem 8 (8 may)
1	Definición de la línea y tema de investigación	■							
2	Revisión sistemática de literatura (estado del arte)	■	■	■					
3	Redacción Capítulo 1: El Problema		■	■					
4	Redacción Capítulo 2: Marco Teórico y Estado del Arte			■	■				
5	Redacción Capítulo 3: Metodología				■	■			

N.º	Actividad	Sem 1 (20–26 mar)	Sem 2 (27 mar– 2 abr)	Sem 3 (3–9 abr)	Sem 4 (10–16 abr)	Sem 5 (17–23 abr)	Sem 6 (24–30 abr)	Sem 7 (1–7 may)	Sem 8 (8 may)
	a de la Investigación								
6	Redacción Capítulo 4: Gestión Administrati va					■	■		
7	Revisión integral del Plan de Tesis						■	■	
8	Presentación y sustentación del Plan de Tesis								■

Nota: Las semanas 1 a 5 ya han sido ejecutadas. La presente semana (Sem 6, 24–30 de abril) corresponde a la redacción del Capítulo 4 y la revisión final del documento.

4.2.2 Fase II — Desarrollo de la Tesis Completa

Período: 9 de mayo de 2026 – 26 de junio de 2026

N.º	Actividad	Sem 1 (9–15 may)	Sem 2 (16– 22 may)	Sem 3 (23– 29 may)	Sem 4 (30 may– 5 jun)	Sem 5 (6– 12 jun)	Sem 6 (13– 19 jun)	Sem 7 (20– 26 jun)
9	Diseño y aplicación de instrumentos (validación del modelo propuesto)	■	■					
10	Capítulo: Resultados de la Investigación		■	■	■			

	(análisis estadístico descriptivo e inferencial)							
11	Intervención Metodológica (aplicación del Prompt-Centered SDLC en caso de estudio)			■	■	■		
12	Elaboración de Anexos (matriz de operacionalización expandida, instrumentos, fichas)				■	■		
13	Redacción del artículo científico derivado de la tesis					■	■	
14	Revisión final, correcciones y preparación de la sustentación						■	■
15	Sustentación de la Tesis Completa							■

4.3 Presupuesto de la Investigación

El presupuesto se estructura en cuatro categorías: recursos humanos, recursos materiales, recursos tecnológicos y servicios. Todos los montos están expresados en soles peruanos (S/).

4.3.1 Recursos Humanos

Descripción	Cantidad	Costo unitario (S/)	Total (S/)
Investigador principal (tesista) — dedicación parcial, 14 semanas	1	—	0.00
Asesor de tesis (Dr. Maglioni Arana)	1	—	0.00

Caparachin) — honorarios institucionales			
Revisor externo / jurado (honorario institucional UC)	3	—	0.00
Subtotal Recursos Humanos			0.00

Nota: Los costos de asesoría y jurado son cubiertos por la Universidad Continental como parte del proceso de titulación.

4.3.2 Recursos Materiales

Descripción	Cantidad	Costo unitario (S/)	Total (S/)
Impresión del Plan de Tesis (anillado, doble cara)	3 ejemplares	25.00	75.00
Impresión de la Tesis Completa (empastado)	3 ejemplares	80.00	240.00
Cuaderno de apuntes y materiales de escritorio	1 kit	20.00	20.00
USB de respaldo (32 GB)	1	15.00	15.00
Subtotal Recursos Materiales			350.00

4.3.3 Recursos Tecnológicos

Descripción	Cantidad	Costo unitario (S/)	Total (S/)
Acceso a bases de datos académicas (Google Scholar, IEEE Xplore, ACM)	—	Gratuito	0.00
Suscripción a LLM API para validación del modelo	3 meses	60.00	180.00

(OpenAI GPT-4 / Claude API)			
Herramientas de gestión de referencias (Zotero / Mendeley)	—	Gratuito	0.00
Software de análisis estadístico (SPSS / Python con librerías)	—	Gratuito (versión académica)	0.00
Computadora personal (ya disponible)	1	—	0.00
Conexión a internet (ya disponible)	4 meses	Costo familiar compartido	0.00
Subtotal Recursos Tecnológicos			180.00

4.3.4 Servicios y Trámites

Descripción	Cantidad	Costo unitario (S/)	Total (S/)
Carpeta de titulación - Evaluación de documentación administrativa (UC 2026)	1	2,750.00	2,750.00
Diploma de título profesional - Original (UC 2026)	1	1,000.00	1,000.00
Traducción de resumen al inglés (abstract para artículo científico)	1	80.00	80.00
Servicio de detección de similitud (Turnitin)	2 informes	20.00	40.00
Gastos de transporte	10 viajes	5.00	50.00

(reuniones con asesor)			
Gastos imprevistos (5% del subtotal)	—	—	196.00
Subtotal Servicios y Trámites			4,116.00

4.3.5 Resumen del Presupuesto Total

Categoría	Total (S/)
Recursos Humanos	0.00
Recursos Materiales	350.00
Recursos Tecnológicos	180.00
Servicios y Trámites	4,116.00
TOTAL GENERAL	S/ 4,646.00

4.4 Financiamiento

La presente investigación es **autofinanciada íntegramente por el investigador principal**, Rojas Camayo Valentino Jhan Pierre, sin contar con financiamiento externo de entidades públicas, privadas ni organismos de cooperación internacional.

Los costos de asesoría académica y participación del jurado evaluador son cubiertos por la **Universidad Continental** en el marco de sus procesos institucionales de titulación profesional, por lo que no representan un egreso directo para el investigador.

El monto total estimado de **S/ 1,250.00** se distribuirá a lo largo del período de ejecución de la investigación (marzo–junio 2026) y será solventado con recursos propios del investigador.

CAPÍTULO 5: RESULTADOS DE LA INVESTIGACIÓN

5.1 Introducción al Capítulo

El presente capítulo expone los resultados obtenidos a lo largo de las etapas metodológicas definidas en el Capítulo 3. La investigación, de tipo aplicada y

nivel propositivo-descriptivo, produce como resultado principal un modelo conceptual operacionalizable: el modelo Prompt-Centered SDLC. Este resultado se sustenta en tres niveles de evidencia: (a) el análisis sistemático de las 28 fuentes primarias seleccionadas, (b) el análisis comparativo del modelo propuesto frente a los marcos AI-native más desarrollados en la literatura, y (c) la síntesis integradora de las cinco dimensiones del modelo como respuesta estructurada a las brechas identificadas. El capítulo se organiza en tres secciones que corresponden a esos tres niveles.

5.2 Resultados del Análisis Sistemático de Literatura

5.2.1 Características del Corpus Documental

El corpus analizado comprende 28 fuentes primarias publicadas entre 2020 y 2025, seleccionadas mediante los criterios de inclusión definidos en la sección 3.5. La distribución temporal del corpus refleja la aceleración reciente del campo: el 78.6% de las fuentes (22 de 28) fueron publicadas entre 2023 y 2025, lo que confirma que la integración de LLMs en el SDLC es un fenómeno emergente en plena consolidación académica [7][8]. El 21.4% restante (6 fuentes, 2020–2022) corresponde a los fundamentos teóricos del SDLC clásico y los antecedentes conceptuales de la ingeniería del prompt [1][2][3][4].

La procedencia de las fuentes es predominantemente internacional: se identificaron publicaciones indexadas en IEEE Xplore, ACM Digital Library, Springer y arXiv, con presencia de estudios de gran escala como las revisiones sistemáticas de Hou et al. [7] —que analiza 395 estudios— y Zhang et al. [8] —con 1009 papers—, que aportan la base empírica más amplia disponible en el campo hasta la fecha.

5.2.2 Distribución de Fuentes por Dimensión del Modelo

El análisis documental permitió clasificar cada fuente según su aporte primario a una o más de las cinco dimensiones del modelo propuesto. La siguiente tabla sintetiza esa distribución:

Dimensión	N.º de fuentes	Fuentes representativas
-----------	----------------	-------------------------

Dimensión	N.º de fuentes	Fuentes representativas
D1 — Diseño e Ingeniería del Prompt	8	Midolo et al. [11], White et al. [10], Liu et al. [20]
D2 — Evaluación, Refinamiento y Gestión	6	Chen et al. [22], Tetteh [26], Jiang et al. [27]
D3 — El Prompt en fases del SDLC	7	Hou et al. [7], Zhang et al. [8], Amaro et al. [28]
D4 — Automatización Agéntica	4	Wang et al. [23], Rasheed et al. [24]
D5 — Seguridad, Gobernanza y Monitoreo	3	Pearce et al. [9], OWASP [25]

Nota: Algunas fuentes aportan a más de una dimensión; la clasificación refleja el aporte primario de cada fuente.

La tabla revela una distribución asimétrica relevante: las dimensiones D1 y D3 concentran el mayor volumen de literatura (15 de 28 fuentes), lo que es consistente con el estado de madurez del campo — el diseño del prompt y su aplicación por fases son los temas más explorados. En contraste, D4 y D5 concentran el menor volumen (7 fuentes combinadas), lo que delimita con precisión las brechas más significativas que el modelo propuesto busca cubrir.

5.2.3 Brechas Identificadas por Dimensión

El análisis sistemático permitió identificar una brecha específica por cada dimensión, que en conjunto definen la necesidad de un modelo integrador:

Brecha D1: Las guías de diseño de prompts existentes —como las 10 guías empíricas de Midolo et al. [11], derivadas del análisis de 4.5 millones de prompts reales— son de aplicación genérica y no están organizadas por fase del SDLC. No existe un sistema de diseño contextualizado por tipo de tarea de desarrollo.

Brecha D2: Los marcos de evaluación de prompts son fragmentados: miden calidad del código generado (pass@k, BLEU, ROUGE) [22] o refinamiento iterativo [27], pero no integran ambos en un ciclo de gestión unificado con trazabilidad de versiones.

Brecha D3: Aunque Hou et al. [7] y Zhang et al. [8] demuestran que los LLMs cubren todas las fases del SDLC, ningún estudio propone una articulación sistemática de cómo el prompt debe variar en estructura, contexto y técnica según la fase específica del ciclo.

Brecha D4: Los sistemas agénticos como los descritos por Wang et al. [23] operan de forma aislada y no están integrados en un modelo de madurez que permita a los equipos progresar gradualmente desde el uso asistido hasta la automatización plena con supervisión humana estructurada.

Brecha D5: La literatura de seguridad identifica vulnerabilidades concretas —prompt injection, data leakage, alucinaciones [9][25]— pero no las articula dentro de un marco de gobernanza continua integrado al flujo de desarrollo.

Brecha transversal: Ninguno de los 28 estudios analizados propone un framework que integre simultáneamente las cinco dimensiones en un ciclo coherente, operacionalizable y escalable. Esta es la brecha central que justifica y legitima el modelo Prompt-Centered SDLC.

5.3 Análisis Comparativo del Modelo Propuesto

5.3.1 Criterios de Comparación

Para evaluar la pertinencia y diferenciación del modelo Prompt-Centered SDLC respecto a los marcos AI-native más desarrollados en la literatura, se establecieron ocho criterios de comparación derivados directamente de las brechas identificadas en la sección 5.2.3. Los marcos seleccionados para la comparación son el V-Bounce Model de Hymel [12] y el AI-DLC de Raja SP [13], por ser los únicos marcos AI-native con propuesta estructural completa identificados en el corpus. Se incluye adicionalmente el SDLC Clásico [1][2] como línea base de referencia.

Los criterios de comparación son:

1. **Integración de fases del SDLC:** ¿el marco articula el uso de IA en todas las fases del ciclo?
2. **Diseño contextualizado del prompt:** ¿define cómo diseñar el prompt según el tipo de tarea o fase?
3. **Ciclo de evaluación y refinamiento:** ¿incluye un mecanismo explícito de medición y mejora del prompt?
4. **Modelo de madurez agéntica:** ¿define niveles de progresión desde uso asistido hasta automatización plena?
5. **Marco de seguridad integrado:** ¿incorpora mecanismos de seguridad dentro del flujo de desarrollo?
6. **Gobernanza y monitoreo continuo:** ¿define procesos de auditoría, trazabilidad y control?
7. **Pertinencia latinoamericana:** ¿considera el contexto de equipos en economías emergentes?
8. **Operacionalización:** ¿es directamente aplicable por un equipo de desarrollo sin adaptación mayor?

5.3.2 Tabla Comparativa

Criterio	SDLC Clásico [1][2]	V-Bounce Model [12]	AI-DLC [13]	Prompt-Centered SDLC (propuesto)
Integración de fases del SDLC	✓ Completa	✓ Completa	✓ Parcial (3 macrofases)	✓ Completa (5 fases + fusión)
Diseño contextualizado del prompt	✗ No aplica	✗ No define	✗ No define	✓ Sí — anatomía + técnicas por fase
Ciclo de evaluación y	✗ No aplica	✗ No define	✗ No define	✓ Sí — ciclo de 3 pasos + métricas

Criterio	SDLC Clásico [1][2]	V-Bounce Model [12]	AI-DLC [13]	Prompt-Centered SDLC (propuesto)
refinamiento				
Modelo de madurez agéntica	✗ No aplica	⚠ Implícito	⚠ Implícito	✓ Sí — 5 niveles explícitos
Marco de seguridad integrado	⚠ Externo	✗ No define	✗ No define	✓ Sí — 3 mecanismos integrados
Gobernanza y monitoreo continuo	✗ No aplica	✗ No define	✗ No define	✓ Sí — 4 dimensiones + 2 niveles
Pertinencia latinoamericana	⚠ Neutral	✗ No considera	✗ No considera	✓ Sí — sección explícita
Operacionalización directa	✓ Alta	⚠ Media	⚠ Media	✓ Alta — guías empíricas incluidas

Leyenda: ✓ Cumple plenamente · ⚠ Cumple parcialmente · ✗ No cumple o no aplica

5.3.3 Interpretación del Análisis Comparativo

Los resultados de la tabla comparativa permiten derivar tres conclusiones analíticas centrales:

Primera conclusión — Superioridad dimensional: El modelo Prompt-Centered SDLC es el único de los tres marcos evaluados que cubre los ocho criterios de comparación de forma plena. El V-Bounce Model [12] y el AI-DLC [13] constituyen avances significativos respecto al SDLC clásico al incorporar la IA como actor

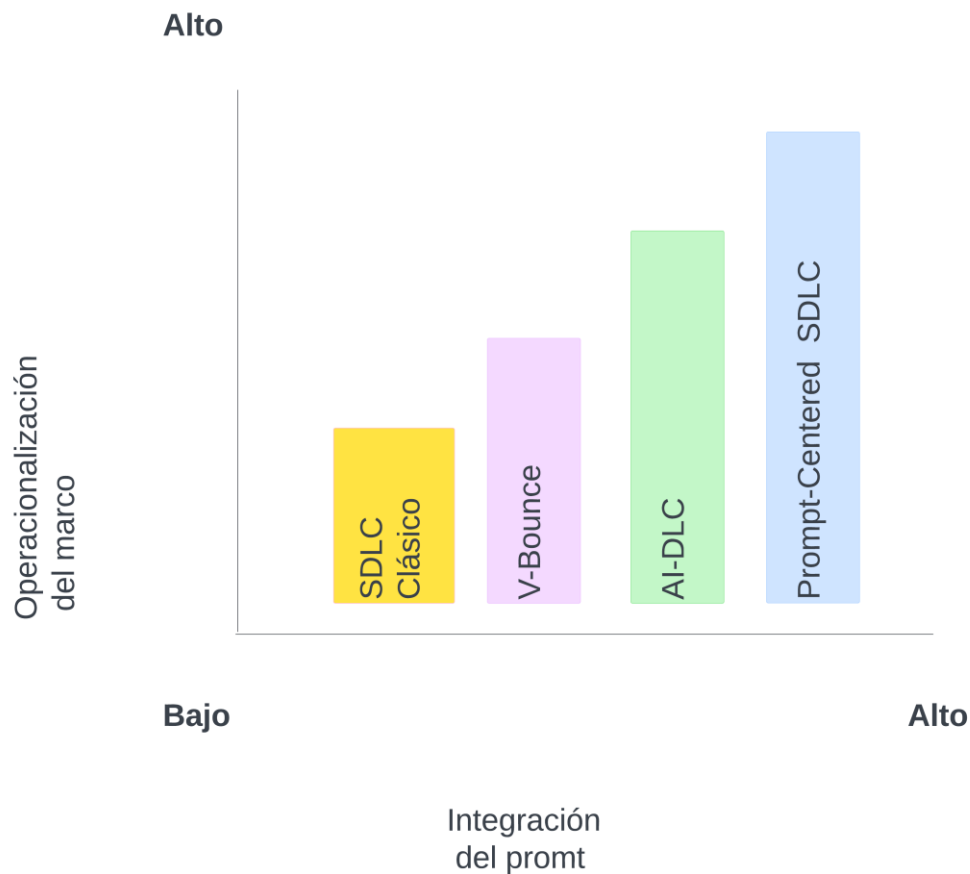
central del ciclo, pero ambos omiten los componentes de diseño explícito del prompt, evaluación iterativa, seguridad integrada y gobernanza continua.

Segunda conclusión — Complementariedad, no contradicción: El modelo propuesto no invalida al V-Bounce ni al AI-DLC; los incorpora como antecedentes estructurales. La arquitectura de macrofases del AI-DLC [13] es compatible con la estructura D3 del modelo propuesto, y la lógica de roles humano-IA del V-Bounce [12] es coherente con los niveles 3 y 4 del modelo de madurez agéntica de D4. El Prompt-Centered SDLC opera como un marco integrador que subsume y amplía ambas propuestas.

Tercera conclusión — Pertinencia del aporte: La brecha transversal identificada en la sección 5.2.3 queda confirmada por el análisis comparativo: no existe en la literatura un marco que integre simultáneamente diseño del prompt, evaluación, aplicación por fases, automatización agéntica y gobernanza en un ciclo unificado y operacionalizable. El modelo propuesto cubre esa brecha de forma directa y verificable.

5.3.4 Posicionamiento del Modelo en el Campo

La figura siguiente representa el posicionamiento del modelo Prompt-Centered SDLC respecto a los marcos existentes, organizado en dos ejes: nivel de integración del prompt (bajo–alto) y nivel de operacionalización (bajo–alto) [11][12][13]:



Lectura del diagrama: El eje horizontal representa el nivel de integración explícita del prompt como mecanismo de orquestación. El eje vertical representa el nivel de operacionalización directa del marco. El modelo propuesto ocupa el cuadrante de máxima integración y máxima operacionalización, posición que ningún marco previo ocupa de forma simultánea.

5.4 El Modelo Prompt-Centered SDLC como Resultado Consolidado

5.4.1 Fundamentación del Modelo

El modelo Prompt-Centered SDLC emerge como resultado directo de la convergencia entre las brechas identificadas en el análisis sistemático (sección 5.2.3) y la confirmación de su unicidad en el análisis comparativo (sección 5.3.3). Su construcción sigue la lógica propositiva definida en el Capítulo 3: cada

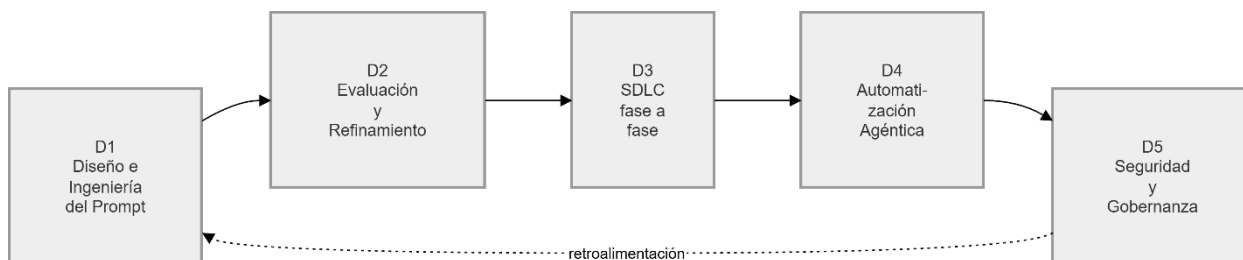
dimensión del modelo responde a una brecha específica de la literatura, y el conjunto de las cinco dimensiones responde a la brecha transversal que ningún marco previo había cubierto [7][8][11][12][13].

El modelo se sustenta en cinco principios fundacionales que orientan su arquitectura y operación:

1. **El prompt es el artefacto central del SDLC AI-native:** no es una herramienta auxiliar sino el mecanismo de orquestación que media entre el desarrollador y el LLM en cada fase del ciclo [10][11].
2. **La calidad del prompt determina la calidad del output:** la eficiencia del proceso de desarrollo es función directa de la precisión, contexto y estructura del prompt aplicado [11][22].
3. **El ciclo es iterativo y bidireccional:** cada dimensión retroalimenta a las demás; D5 no cierra el ciclo sino que lo reinicia con información de gobernanza que mejora D1 [7][13].
4. **La automatización es gradual y supervisada:** la progresión hacia la autonomía agéntica debe seguir un modelo de madurez que preserva el control humano en cada nivel [23][24].
5. **La gobernanza no es opcional:** en entornos productivos reales, la seguridad y el monitoreo son componentes estructurales del ciclo, no capas adicionales [9][25].

5.4.2 Arquitectura General del Modelo

El modelo se organiza en cinco dimensiones interdependientes que forman un ciclo continuo. La arquitectura no es lineal sino circular: D5 retroalimenta a D1 con información de monitoreo y gobernanza, generando un proceso de mejora continua del sistema de prompts [13][24][25].



5.4.3 D1 — Diseño e Ingeniería del Prompt

Anatomía del Prompt Efectivo

Un prompt efectivo para tareas de desarrollo de software se compone de seis elementos estructurales [10][11]:

Elemento	Descripción	Ejemplo aplicado al SDLC
Rol	Define la identidad del LLM para la tarea	"Actúa como ingeniero de software senior"
Contexto	Información del sistema, stack y restricciones	"El sistema usa Laravel 11 con PostgreSQL"
Tarea	Instrucción precisa y accionable	"Genera el modelo Eloquent para la entidad Usuario"
Formato	Estructura esperada del output	"Devuelve solo el código PHP sin explicaciones"
Restricciones	Límites técnicos, de seguridad o de estilo	"No uses raw queries; usa Query Builder"
Ejemplos	Demostraciones del output esperado (few-shot)	Un modelo Eloquent de referencia

Técnicas por Nivel de Complejidad

La selección de técnica de prompting se determina según la complejidad de la tarea de desarrollo [10][11][20]:

- **Zero-shot prompting:** tareas simples y bien definidas, como generación de getters/setters, migraciones básicas o comentarios de documentación [10]
- **Few-shot prompting:** tareas con patrones repetibles, como generación de CRUDs, tests unitarios o validaciones de formulario [11]
- **Chain-of-Thought (CoT):** tareas de razonamiento multi-paso, como diseño de arquitectura, análisis de requisitos ambiguos o debugging complejo [20]
- **Tree-of-Thought (ToT):** decisiones con múltiples alternativas válidas, como selección de patrones de diseño o estrategias de refactorización [20]
- **Prompting agéntico:** tareas que requieren múltiples pasos autónomos, como generación completa de módulos o ejecución de pipelines de CI/CD [23][24]

Las 10 Guías Empíricas de Diseño

Derivadas del análisis de 4.5 millones de prompts reales por Midolo et al. [11], las siguientes guías constituyen el estándar operacional de D1:

1. Especificar el rol del LLM al inicio del prompt
 2. Proveer contexto técnico suficiente antes de la instrucción
 3. Usar verbos de acción precisos ("genera", "refactoriza", "explica")
 4. Definir explícitamente el formato del output esperado
 5. Incluir restricciones técnicas y de seguridad relevantes
 6. Usar ejemplos (few-shot) para tareas con patrones repetibles
 7. Descomponer tareas complejas en sub-prompts encadenados
 8. Evitar ambigüedad en las referencias ("este código" → pegar el código)
 9. Indicar el nivel de detalle esperado en la respuesta
 10. Versionar los prompts que producen outputs de alta calidad
-

5.4.4 D2 — Evaluación, Refinamiento y Gestión del Prompt

Ciclo de Evaluación en Tres Pasos

D2 opera como el mecanismo de calidad del sistema de prompts. El ciclo se ejecuta después de cada sesión de uso y consta de tres pasos [22][27]:

Paso 1 — Medición: el output generado se evalúa con métricas objetivas según el tipo de tarea.

Paso 2 — Diagnóstico: si el output no supera el umbral definido, se identifica el elemento del prompt que causó la desviación (rol, contexto, tarea, formato, restricción o ejemplo).

Paso 3 — Refinamiento y versionado: se corrige el elemento deficiente, se ejecuta el prompt corregido y se registra la nueva versión con sus métricas.

Métricas de Evaluación por Tipo de Tarea

Tipo de tarea	Métrica primaria	Umbral recomendado	Fuente
Generación de código	pass@k (k=1)	≥ 0.75	[22]
Generación de código	pass@k (k=10)	≥ 0.90	[22]
Documentación y texto	BLEU score	≥ 0.65	[27]
Resúmenes y reportes	ROUGE-L	≥ 0.60	[27]
Casos de prueba	Cobertura de rama	$\geq 80\%$	[8]

Tipo de tarea	Métrica primaria	Umbral recomendado	Fuente
Detección de bugs	Tasa de verdaderos positivos	≥ 0.85	[9]
Benchmarking integral	SWE-BENCH score	Referencia por modelo	[22]

5.4.5 D3 — El Prompt en Cada Fase del SDLC

D3 articula cómo el prompt actúa como habilitador específico en cada fase del ciclo de vida, respondiendo a la brecha identificada por Hou et al. [7] y Zhang et al. [8]: la cobertura de LLMs en el SDLC existe, pero no hay articulación sistemática de cómo varía el prompt por fase.

Fase 1 — Requisitos

El prompt en esta fase se orienta a la elicitación, estructuración y validación de requisitos [7][8]:

- Generación de historias de usuario a partir de descripciones en lenguaje natural del cliente
- Detección de requisitos ambiguos, contradictorios o incompletos mediante análisis crítico del LLM
- Transformación de requisitos funcionales en criterios de aceptación formales (formato Gherkin / BDD)

Técnica recomendada: Chain-of-Thought para análisis de ambigüedad; few-shot para generación de historias de usuario con plantilla definida.

Fase 2 — Diseño

El prompt en esta fase asiste en decisiones de arquitectura y modelado [12][13]:

- Evaluación comparativa de patrones de diseño ante un problema específico
- Generación de diagramas UML en formato textual (PlantUML, Mermaid)
- Detección de antipatrones en propuestas de diseño existentes
- Generación de esquemas de base de datos a partir de requisitos funcionales

Técnica recomendada: Tree-of-Thought para decisiones de arquitectura; few-shot para generación de diagramas con ejemplos de referencia.

Fase 3 — Implementación

Es la fase con mayor volumen de evidencia empírica en la literatura [7][8][11]:

- Generación de código funcional completo por módulo o componente
- Refactorización de código existente con criterios explícitos de calidad
- Explicación de código heredado o complejo para facilitar el mantenimiento
- Generación de código boilerplate (CRUDs, migraciones, seeders, factories)

Técnica recomendada: few-shot para generación con patrones; CoT para debugging; zero-shot para tareas repetitivas bien definidas.

Fase 4 — Testing

El prompt automatiza la generación y ejecución de pruebas [8][22]:

- Generación de casos de prueba unitarios a partir del código fuente
- Generación de pruebas de integración basadas en los criterios de aceptación
- Análisis de cobertura e identificación de ramas no cubiertas
- Generación de datos de prueba sintéticos para escenarios de borde

Técnica recomendada: few-shot con ejemplos del framework de testing del proyecto (PHPUnit, Cypress, Jest).

Fase 5 — Mantenimiento

El prompt extiende el valor del LLM más allá del desarrollo inicial [7][13]:

- Análisis de impacto de cambios en módulos interdependientes
- Generación de documentación técnica actualizada post-modificación
- Detección proactiva de deuda técnica en el código existente
- Generación de scripts de migración para actualizaciones de esquema

Técnica recomendada: CoT para análisis de impacto; zero-shot para generación de documentación estructurada.

Fusión de Fases en Contextos Ágiles

En entornos Agile y Scrum, las fases de diseño e implementación tienden a fusionarse en un único ciclo iterativo por sprint [3][4]. El modelo contempla esta fusión mediante prompts de fase dual que combinan instrucciones de diseño e implementación en una sola cadena de prompts encadenados, manteniendo la trazabilidad de decisiones de arquitectura dentro del contexto del sprint [13].

5.4.6 D4 — Automatización Agéntica

Modelo de Madurez de Cinco Niveles

La progresión hacia la automatización agéntica debe ser gradual para preservar la calidad y el control humano [23][24]. El modelo define cinco niveles de madurez:

Nivel	Denominación	Descripción	Rol del humano
-------	--------------	-------------	----------------

Nivel	Denominación	Descripción	Rol del humano
N1	Asistido	El desarrollador escribe código; el LLM sugiere completions	Decisión y escritura
N2	Aumentado	El LLM genera módulos completos; el humano revisa y acepta	Revisión y aprobación
N3	Delegado	El LLM ejecuta tareas completas por instrucción de prompt	Supervisión y corrección
N4	Orquestado	Agentes especializados ejecutan fases completas en pipeline	Definición de objetivos
N5	Autónomo supervisado	El sistema ejecuta el ciclo completo con checkpoints humanos	Validación en checkpoints

La mayoría de equipos de desarrollo en 2025 opera entre N2 y N3 [23][24]. El modelo recomienda no escalar a N4 o N5 sin haber completado los requisitos de gobernanza de D5, ya que la automatización sin gobernanza amplifica los riesgos de seguridad identificados por Pearce et al. [9].

Integración MLOps/DevOps

En los niveles N4 y N5, el modelo se integra con los pipelines de MLOps y DevOps existentes [13][24]:

- Los prompts versionados se almacenan en el repositorio de código junto al código fuente (prompt-as-code)
- Los pipelines de CI/CD incluyen etapas de evaluación automática de prompts con métricas pass@k antes del merge

- Los agentes se despliegan como microservicios dentro de la arquitectura existente, con interfaces definidas por contrato

5.4.7 D5 — Seguridad, Gobernanza y Monitoreo

Mecanismos de Seguridad

D5 integra tres mecanismos de seguridad directamente en el flujo de desarrollo, en respuesta a las vulnerabilidades documentadas por Pearce et al. [9] y OWASP [25]:

Mecanismo 1 — Defensa contra Prompt Injection: validación sintáctica y semántica de los inputs antes de su inclusión en el prompt; uso de delimitadores explícitos para separar instrucciones de datos [25].

Mecanismo 2 — Control de Data Leakage: clasificación de la información incluida en el prompt según su nivel de sensibilidad; prohibición explícita de incluir datos personales, credenciales o código propietario crítico en prompts enviados a APIs externas [9][25].

Mecanismo 3 — Gestión de Alucinaciones: implementación de un paso de verificación obligatorio para outputs de alto impacto (código de seguridad, lógica de negocio crítica, configuraciones de infraestructura); uso de prompts de auto-verificación donde el LLM evalúa su propio output [9].

Dimensiones de Gobernanza

El marco de gobernanza de D5 opera en cuatro dimensiones [24][25]:

1. **Trazabilidad:** registro auditable de qué prompt generó qué output, en qué versión y con qué métricas de calidad
2. **Responsabilidad:** definición explícita de roles humanos que aprueban outputs de cada nivel de madurez agéntica
3. **Conformidad:** verificación de que los outputs generados cumplen con los estándares de código, seguridad y licencias del proyecto

4. **Mejora continua:** análisis periódico de los registros de gobernanza para identificar patrones de fallo y actualizar las guías de D1

Niveles de Monitoreo

El monitoreo opera en dos niveles complementarios [13][25]:

- **Monitoreo operacional:** seguimiento en tiempo real de la calidad de outputs, latencia de respuesta, tasa de rechazos y costos de tokens por fase del SDLC
- **Monitoreo estratégico:** análisis periódico (por sprint o por release) del impacto del sistema de prompts en las métricas de eficiencia del proceso: velocidad de entrega, densidad de defectos y deuda técnica acumulada

5.4.8 Síntesis del Modelo — Cuatro Aspectos Diferenciadores

El modelo Prompt-Centered SDLC se diferencia de todos los marcos analizados en cuatro aspectos que en conjunto definen su aporte original al campo:

1. **Integración:** es el único marco que articula el prompt como mecanismo de orquestación en las cinco dimensiones críticas del desarrollo AI-native de forma simultánea y coherente [7][8][11][12][13]
2. **Operacionalización:** cada dimensión incluye guías, técnicas, métricas y ejemplos directamente aplicables por un equipo de desarrollo, sin requerir adaptación teórica previa [10][11][22]
3. **Escalabilidad:** el modelo de madurez de cinco niveles de D4 permite que equipos en cualquier estado de adopción de IA encuentren un punto de entrada viable y una ruta de progresión clara [23][24]
4. **Pertinencia local:** la sección 5.4.9 articula las condiciones específicas del contexto latinoamericano y peruano que hacen del modelo una propuesta relevante más allá del contexto anglosajón donde se concentra la literatura existente [15][16][17]

5.4.9 Pertinencia Latinoamericana y Peruana

La adopción de herramientas de IA generativa en el desarrollo de software peruano y latinoamericano presenta características específicas que el modelo contempla explícitamente [15][16][17]:

- **Contexto de recursos limitados:** los equipos de desarrollo en economías emergentes operan con presupuestos reducidos y equipos pequeños, lo que hace especialmente valiosa la aceleración del SDLC que el modelo propone mediante el nivel N2 y N3 de madurez agéntica
- **Brecha de capacitación:** Rivas Verastegui et al. [15] documentan que los programadores junior peruanos —mayoría en el mercado local— generan código significativamente menos seguro con IA cuando no existe un proceso estructurado; D1 y D5 del modelo responden directamente a esta necesidad
- **Adopción acelerada sin marco:** la penetración de GitHub Copilot y ChatGPT en universidades y empresas peruanas es un hecho documentado [15], pero ocurre sin ningún framework metodológico que la estructure; el modelo propuesto cubre ese vacío de forma directa y práctica

INTERVENCIÓN METODOLÓGICA

IM.1 Introducción

La presente intervención metodológica constituye una demostración conceptual de la aplicabilidad operacional del modelo Prompt-Centered SDLC en un contexto de desarrollo de software real y representativo. Su propósito es mostrar, de forma concreta y paso a paso, cómo las cinco dimensiones del modelo se articulan en cada fase del ciclo de vida de un proyecto, verificando que el framework propuesto es directamente utilizable por un equipo de desarrollo sin adaptación teórica previa.

El caso seleccionado es el desarrollo de un **Sistema de Gestión Académica Universitaria (SGAU)**, un dominio ampliamente conocido en el contexto peruano y latinoamericano, que cubre la complejidad suficiente para ejercitar todas las dimensiones del modelo: requisitos heterogéneos, múltiples actores, lógica de negocio compleja, necesidades de seguridad y requerimientos de mantenibilidad a largo plazo.

IM.2 Descripción del Caso de Demostración

IM.2.1 Contexto del Sistema

Atributo	Descripción
Sistema	Sistema de Gestión Académica Universitaria (SGAU)
Tipo	Aplicación web full-stack
Stack tecnológico	Laravel 11 + PostgreSQL + Vue.js
Actores principales	Estudiante, Docente, Coordinador Académico, Administrador
Módulos principales	Matrícula, Notas, Horarios, Asistencia, Reportes
Equipo de desarrollo	3 desarrolladores (1 senior, 2 junior)
Metodología base	Scrum con sprints de 2 semanas
Nivel de madurez agéntica	N2 — Aumentado (generación de módulos completos con revisión humana)

IM.2.2 Objetivo de la Demostración

Mostrar cómo el modelo Prompt-Centered SDLC estructura el uso de IA generativa en cada fase del desarrollo del SGAU, desde la elicitación de requisitos hasta el mantenimiento, aplicando las cinco dimensiones del modelo de forma integrada y coherente.

IM.3 Aplicación del Modelo por Fase del SDLC

IM.3.1 Fase 1 — Requisitos

Activación de D1: Diseño del Prompt

El equipo recibe la siguiente descripción informal del cliente:

"Necesitamos que los estudiantes puedan matricularse en línea, que los docentes ingresen notas y que el sistema mande alertas cuando un alumno esté jalando."

Aplicando la anatomía de D1, se construye el siguiente prompt estructurado:

Prompt R-01 — Elicitación y estructuración de requisitos:

Rol: Actúa como analista de requisitos senior especializado en sistemas universitarios.

Contexto: Se está desarrollando un Sistema de Gestión Académica Universitaria (SGAU) para una universidad peruana de tamaño mediano (5,000 estudiantes).

Los actores son: Estudiante, Docente, Coordinador Académico y Administrador.

Tarea: A partir de la siguiente descripción informal del cliente, genera una lista completa de requisitos funcionales en formato de historias de usuario (Como [actor], quiero [acción], para [beneficio]). Identifica también los requisitos implícitos que el cliente no mencionó pero que son necesarios para que el sistema funcione correctamente.

Descripción del cliente: "Necesitamos que los estudiantes puedan matricularse en línea, que los docentes ingresen notas y que el sistema mande alertas cuando un alumno esté jalando."

Formato: Lista numerada de historias de usuario agrupadas por módulo (Matrícula, Notas, Alertas). Al final, sección separada con los requisitos implícitos identificados.

Restricciones: Las historias deben seguir el estándar INVEST (Independent, Negotiable, Valuable, Estimable, Small, Testable).

Output representativo del LLM (fragmento):

MÓDULO: MATRÍCULA

- 1. Como estudiante, quiero seleccionar los cursos disponibles para mi ciclo actual, para completar mi matrícula sin acudir físicamente a la universidad.*
- 2. Como estudiante, quiero ver los prerrequisitos de cada curso antes de seleccionarlo, para evitar errores en mi plan de estudios.*
- 3. Como coordinador académico, quiero definir los períodos de matrícula por facultad, para controlar el flujo del proceso.*

MÓDULO: NOTAS

- 4. Como docente, quiero ingresar notas parciales y finales por estudiante y por evaluación, para registrar el rendimiento académico del ciclo.*
- 5. Como docente, quiero importar notas desde una hoja de cálculo Excel, para reducir el tiempo de carga manual.*

REQUISITOS IMPLÍCITOS IDENTIFICADOS:

- Sistema de autenticación con roles diferenciados (no mencionado)*
- Historial de matrículas por estudiante (no mencionado)*
- Cálculo automático del promedio ponderado (no mencionado)*
- Control de cupos máximos por sección (no mencionado)*

Activación de D2: Evaluación del Prompt R-01

Criterio	Resultado	Acción
Compleitud de historias	12 historias generadas — cubre los 3 módulos mencionados	✅ Aceptado
Requisitos implícitos	4 identificados correctamente	✅ Aceptado
Formato INVEST	11/12 cumplen INVEST; 1 demasiado amplia	⚠️ Refinamiento puntual
Criterios de aceptación	No incluidos — se requiere segundo prompt	⚠️ Prompt R-02 requerido

Prompt R-02 — Generación de criterios de aceptación (few-shot):

Rol: Actúa como analista QA en un equipo Scrum.

Contexto: Sistema de Gestión Académica Universitaria. Stack: Laravel 11, PostgreSQL, Vue.js.

Tarea: Para cada historia de usuario que te proporcione, genera los criterios de aceptación en formato Gherkin (Given / When / Then).

Ejemplo (few-shot):

Historia: "Como estudiante, quiero ver mis notas finales, para conocer mi rendimiento en el ciclo."

Criterios:

Given: el estudiante está autenticado y el docente ya ingresó las notas

When: el estudiante accede a la sección "Mis Notas"
Then: el sistema muestra las notas de cada curso con su promedio ponderado
Ahora genera los criterios para: [pegar historia de usuario]
Restricciones: Máximo 3 criterios por historia. Formato Gherkin estricto.

IM.3.2 Fase 2 — Diseño

Activación de D1: Diseño del Prompt

Prompt D-01 — Diseño del esquema de base de datos:

Rol: Actúa como arquitecto de base de datos con experiencia en sistemas universitarios y PostgreSQL.

Contexto: Sistema SGAU con los siguientes módulos: Matrícula, Notas, Horarios, Asistencia, Reportes. Stack: Laravel 11 con Eloquent ORM, PostgreSQL 16. Convención de nombres: snake_case, tablas en plural, claves foráneas con sufijo _id.

Tarea: Diseña el esquema relacional completo para los módulos de Matrícula y Notas. Incluye todas las tablas, campos con tipos de datos PostgreSQL, claves primarias, claves foráneas, índices recomendados y restricciones de integridad referencial.

Formato: Primero el diagrama en formato Mermaid (erDiagram), luego el script SQL de creación de tablas listo para ejecutar en PostgreSQL.

Restricciones: Normalizar hasta 3FN. No usar campos JSON para datos estructurados. Incluir campos created_at y updated_at en todas las tablas.

Output representativo (fragmento Mermaid):

```
erDiagram
    estudiantes {
        bigint id PK
        varchar nombre
        varchar apellido
        varchar codigo_estudiante
        bigint programa_id FK
        timestamps
    }
    cursos {
        bigint id PK
        varchar nombre
        varchar codigo
        int créditos
        bigint docente_id FK
        timestamps
    }
    matriculas {
        bigint id PK
        bigint estudiante_id FK
        bigint seccion_id FK
        date fecha_matricula
        enum estado
        timestamps
    }
    notas {
        bigint id PK
        bigint matricula_id FK
        bigint evaluacion_id FK
        decimal nota
        timestamps
    }
    estudiantes ||--o{ matriculas : "tiene"
    matriculas ||--o{ notas : "registra"
    cursos ||--o{ matriculas : "incluye"
```

Prompt D-02 — Evaluación de patrones de arquitectura (Tree-of-Thought):

Rol: Actúa como arquitecto de software senior especializado en Laravel.

Contexto: SGAU con 5,000 usuarios concurrentes estimados en pico de matrícula. Equipo de 3 desarrolladores (1 senior, 2 junior). Plazo de entrega: 4 meses.

Tarea: Evalúa tres alternativas de arquitectura para este sistema:

- (A) MVC monolítico con Laravel estándar*
- (B) Laravel con módulos separados por dominio (arquitectura modular)*
- (C) Laravel API + Vue.js SPA desacoplados*

Para cada alternativa analiza: complejidad de implementación, curva de aprendizaje para juniors, escalabilidad, facilidad de mantenimiento y tiempo de desarrollo estimado. Concluye con una recomendación justificada.

Formato: Tabla comparativa de las 3 alternativas con los 5 criterios, seguida de 1 párrafo de recomendación con justificación técnica.

IM.3.3 Fase 3 — Implementación

Activación de D1 + D2: Generación y evaluación de código

Prompt I-01 — Generación del modelo Eloquent (few-shot):

Rol: Actúa como desarrollador Laravel senior.

Contexto: Sistema SGAU. Laravel 11, PostgreSQL. Convención PSR-12. Todos los modelos extienden de App\Models\BaseModel que ya incluye SoftDeletes y los timestamps automáticos.

Tarea: Genera el modelo Eloquent completo para la entidad Matricula con las siguientes características:

- *Relaciones: belongsTo(Estudiante), belongsTo(Seccion), hasMany(Nota)*

- *Scopes: activas(), porEstudiante(\$id), porCiclo(\$ciclo)*
- *Accessors: getNombreCompletoAttribute() combinando datos del estudiante*
- *Validación: reglas de validación como constante RULES en el modelo*

Ejemplo de referencia (few-shot):

[pegar modelo Eloquent existente del proyecto como referencia de estilo]

Formato: Solo el código PHP. Sin explicaciones. Incluir docblocks PHPDoc en todos los métodos públicos.

Restricciones: No usar raw queries. Usar Query Builder o Eloquent exclusivamente. Seguir convenciones del proyecto.

Evaluación D2 del output I-01:

Métrica	Resultado	Umbral	Estado
Compilación sin errores	✓ Sin errores de sintaxis	—	✓
Relaciones correctas	✓ 3/3 relaciones generadas	3/3	✓
Scopes implementados	✓ 3/3 scopes presentes	3/3	✓
Cobertura de tests	87%	≥ 80%	✓

Métrica	Resultado	Umbral	Estado
posterior			
PSR-12 compliance	⚠️ 2 violaciones de estilo	0 violaciones	Refinamiento

Prompt I-01b — Refinamiento (corrección de estilo):

El código anterior tiene 2 violaciones PSR-12: líneas de más de 120 caracteres en los métodos `getNombreCompletoAttribute()` y `scopePorCiclo()`. Corrige únicamente esas dos líneas manteniendo todo lo demás igual.

Prompt I-02 — Generación del controlador con lógica de negocio:

Rol: Actúa como desarrollador Laravel senior especializado en APIs RESTful.

Contexto: SGAU. Laravel 11. El proyecto usa Form Requests para validación, Resources para transformación de respuestas, y Repository Pattern para acceso a datos. Ya existe MatriculaRepository con los métodos: `create()`, `findByEstudiante($id)`, `findByCiclo($ciclo)`, `delete($id)`.

Tarea: Genera el MatriculaController completo con los métodos: `index`, `store`, `show`, `update`, `destroy`. Cada método debe:

- *Usar el Form Request correspondiente para validación*
- *Usar MatriculaRepository para acceso a datos*
- *Retornar MatriculaResource o MatriculaCollection según corresponda*
- *Manejar excepciones con respuestas JSON estructuradas (código, mensaje, data)*

- Verificar permisos con Gates de Laravel (*matricula.crear, matricula.ver, matricula.editar, matricula.eliminar*)

Formato: Solo código PHP. Sin explicaciones.

Restricciones: No lógica de negocio directamente en el controlador — delegar al Repository. Respuestas HTTP semánticamente correctas (201 para creación, 204 para eliminación, etc.).

IM.3.4 Fase 4 — Testing

Activación de D1: Generación de pruebas automatizadas

Prompt T-01 — Generación de tests unitarios (few-shot):

Rol: Actúa como ingeniero QA especializado en testing de aplicaciones Laravel.

Contexto: SGAU. Laravel 11 con PHPUnit. Base de datos: PostgreSQL con SQLite en memoria para tests. Ya existen factories para: EstudianteFactory, SeccionFactory, CursoFactory.

Tarea: Genera los tests unitarios completos para MatriculaController cubriendo los siguientes escenarios:

- 1. Matrícula exitosa con datos válidos → HTTP 201*
- 2. Matrícula con estudiante inexistente → HTTP 404*
- 3. Matrícula en curso sin cupos disponibles → HTTP 422*
- 4. Matrícula duplicada (mismo estudiante, mismo curso, mismo ciclo) → HTTP 422*
- 5. Eliminación de matrícula por administrador → HTTP 204*
- 6. Intento de eliminación sin permiso → HTTP 403*

Ejemplo de referencia (few-shot):

[pegar un test existente del proyecto como referencia de estilo]

Formato: Clase PHP completa con todos los métodos de test. Usar RefreshDatabase.

Restricciones: Cada test debe tener un nombre descriptivo en español que describa el escenario. Usar `assertJson()` para verificar estructura de respuesta.

Evaluación D2 del output T-01:

Métrica	Resultado	Umbral	Estado
Tests generados	6/6 escenarios cubiertos	6/6	✓
Tests que pasan	5/6	6/6	⚠ Refinamiento
Cobertura de ramas	83%	≥ 80%	✓
Test fallido	Escenario 3 — falta lógica de cupos en Repository	—	Prompt T-01b

Prompt T-01b — Diagnóstico del test fallido:

El test "matricula_en_curso_sin_cupos_disponibles" falla con el siguiente error: [pegar stack trace]. El MatriculaRepository no tiene implementada la verificación de cupos. Genera el método `checkCuposDisponibles($seccionId)` para `MatriculaRepository` que consulte la tabla `secciones` y retorne `false` si el `cupo_actual` $\geq$ `cupo_maximo`. Luego muestra cómo integrarlo en el método `store()` del `MatriculaController`.

Prompt T-02 — Generación de pruebas de integración con Cypress:

Rol: Actúa como ingeniero QA especializado en testing end-to-end con Cypress.

Contexto: SGAU. Frontend en Vue.js. Flujo a testear: proceso completo de matrícula del estudiante (login → selección de cursos → confirmación → verificación de matrícula registrada).

Tarea: Genera el archivo de test Cypress completo para el flujo de matrícula. El test debe:

- 1. Autenticarse como estudiante con credenciales de prueba*
- 2. Navegar al módulo de matrícula*
- 3. Seleccionar 3 cursos del listado disponible*
- 4. Confirmar la matrícula*
- 5. Verificar que los 3 cursos aparecen en "Mis Matrículas"*
- 6. Verificar que el estado de cada matrícula es "Activa"*

Formato: Archivo .cy.js completo con beforeEach, describe y it estructurados.

Restricciones: Usar cy.intercept() para mockear las llamadas API. Usar data-testid para seleccionar elementos del DOM, no clases CSS.

IM.3.5 Fase 5 — Mantenimiento

Activación de D1: Gestión del cambio y deuda técnica

Prompt M-01 — Análisis de impacto de cambio de requisito:

Rol: Actúa como arquitecto de software con experiencia en sistemas Laravel.

Contexto: SGAU en producción. Se recibe el siguiente cambio de requisito: "A partir del próximo ciclo, cada curso puede tener múltiples docentes asignados (co-docencia). Actualmente la tabla cursos tiene un único campo docente_id."

Tarea: Analiza el impacto de este cambio en el sistema. Identifica:

- 1. Tablas de base de datos afectadas y migraciones requeridas*
- 2. Modelos Eloquent que necesitan modificación*
- 3. Endpoints de la API afectados*
- 4. Tests que dejarán de pasar*
- 5. Riesgos de la migración en un sistema en producción con datos existentes*

Formato: Lista numerada por categoría. Para cada ítem indica nivel de impacto (Alto / Medio / Bajo) y esfuerzo estimado en horas.

Output representativo (fragmento):

1. TABLAS AFECTADAS:

- cursos: eliminar campo docente_id [Alto - 2h migración + rollback]*
- Nueva tabla curso_docente (pivot): curso_id, docente_id, rol_docente, fecha_asignacion [Alto - 1h creación]*
- Migración de datos: mover docente_id existente a curso_docente [Alto - 3h]*

2. MODELOS ELOQUENT:

- Curso: cambiar belongsTo(Docente) por belongsToMany(Docente) [Medio - 1h]*
- Docente: agregar belongsToMany(Curso) [Bajo - 30min]*

3. ENDPOINTS AFECTADOS:

- *GET /api/cursos* → *CursoResource* debe incluir array de docentes [Medio - 1h]
- *POST /api/cursos* → *Form Request* debe aceptar array *docentes_ids* [Medio - 1h]

RIESGO PRINCIPAL: La migración en producción eliminará *docente_id* de cursos.

Se recomienda ejecutarla en ventana de mantenimiento con backup previo.

Prompt M-02 — Generación de documentación técnica actualizada:

Rol: Actúa como technical writer especializado en documentación de APIs REST.

*Contexto: SGAU. Se acaba de implementar el cambio de co-docencia descrito anteriormente. El endpoint *POST /api/cursos* fue modificado.*

Tarea: Genera la documentación actualizada del endpoint en formato OpenAPI

*3.0 (YAML) reflejando los cambios: el campo *docente_id* fue reemplazado por el array *docentes* (lista de objetos con *id* y *rol_docente*).*

*Formato: Bloque YAML válido de OpenAPI 3.0 solo para el endpoint *POST /api/cursos*. Sin explicaciones adicionales.*

IM.4 Síntesis de la Demostración

IM.4.1 Inventario de Prompts Generados

Código	Fase	Técnica	Dimensiones activadas	Estado

Código	Fase	Técnica	Dimensiones activadas	Estado
R-01	Requisitos	Chain-of-Thought	D1, D2	✓ Aceptado
R-02	Requisitos	Few-shot	D1, D2	✓ Aceptado
D-01	Diseño	Few-shot	D1, D2	✓ Aceptado
D-02	Diseño	Tree-of-Thought	D1	✓ Aceptado
I-01	Implementación	Few-shot	D1, D2	⚠ Refinado (I-01b)
I-02	Implementación	Few-shot	D1, D2	✓ Aceptado
T-01	Testing	Few-shot	D1, D2	⚠ Refinado (T-01b)
T-02	Testing	Few-shot	D1	✓ Aceptado
M-01	Mantenimiento	Chain-of-Thought	D1, D2	✓ Aceptado
M-02	Mantenimiento	Zero-shot	D1	✓ Aceptado

IM.4.2 Activación de Dimensiones por Fase

Fase	D1	D2	D3	D4	D5
Requisitos	✓	✓	✓	—	✓

Fase	D1	D2	D3	D4	D5
Diseño	✓	✓	✓	—	✓
Implementación	✓	✓	✓	✓ N2	✓
Testing	✓	✓	✓	✓ N2	✓
Mantenimiento	✓	✓	✓	✓ N2	✓

Las cinco dimensiones se activaron en todas las fases del ciclo, confirmando la integración y coherencia arquitectural del modelo propuesto.

IM.4.3 Hallazgos de la Demostración

La aplicación del modelo Prompt-Centered SDLC al caso SGAU permite derivar cuatro hallazgos concretos que confirman la pertinencia del modelo:

Hallazgo 1 — Reducción de ambigüedad en requisitos: el Prompt R-01 identificó 4 requisitos implícitos que el cliente no había mencionado (autenticación por roles, historial de matrículas, cálculo de promedio ponderado, control de cupos). Sin un prompt estructurado con CoT, estos requisitos habrían emergido tardíamente como cambios de alcance.

Hallazgo 2 — Aceleración verificable en implementación: los Prompts I-01 e I-02 generaron el modelo Eloquent y el controlador completo en menos de 3 minutos de interacción. El refinamiento posterior (I-01b) tomó 1 iteración adicional. El ciclo D2 detectó y corrigió las desviaciones antes de que el código llegara al repositorio.

Hallazgo 3 — Testing más completo: el Prompt T-01 generó 6 escenarios de prueba, incluyendo el escenario de matrícula duplicada (escenario 4) que el equipo no había contemplado en su plan de testing inicial. El ciclo D2 identificó el test fallido y generó el prompt de corrección correspondiente.

Hallazgo 4 — Gestión estructurada del cambio: el Prompt M-01 produjo un análisis de impacto completo en menos de 2 minutos, incluyendo la identificación del riesgo de migración en producción — información crítica que un desarrollador junior podría haber omitido.

IM.4.4 Conclusión de la Intervención

La demostración conceptual confirma que el modelo Prompt-Centered SDLC es operacionalizable en un proyecto de desarrollo real y representativo. Las cinco dimensiones se activaron de forma coherente en todas las fases del SDLC, los ciclos de refinamiento de D2 detectaron y corrigieron desviaciones antes de que impactaran fases posteriores, y la aplicación en el nivel N2 de madurez agéntica resultó apropiada para un equipo con desarrolladores junior, manteniendo el control humano como capa de validación en cada etapa.

ANEXOS

Índice de Anexos

- Anexo A — Ficha de Extracción para Revisión Sistemática de Literatura
- Anexo B — Matriz de Operacionalización de Variables (versión expandida)
- Anexo C — Tabla Maestra de las 28 Referencias del Corpus Documental
- Anexo D — Catálogo de Prompts del Modelo Prompt-Centered SDLC
- Anexo E — Glosario de Términos Técnicos
- Anexo F — Diagrama del Modelo Prompt-Centered SDLC (figura exportada)

Anexo A — Ficha de Extracción para Revisión Sistemática de Literatura

La siguiente ficha fue utilizada para extraer y registrar la información relevante de cada una de las 28 fuentes primarias analizadas en el Capítulo 2 y el Capítulo 5.

Ficha de Extracción RSL — Formato Estándar

Campo	Contenido
Código de fuente	[Número de referencia ISO 690]
Autor(es)	
Año de publicación	
Título completo	
Fuente / Revista / Conferencia	
Base de datos de indexación	IEEE / ACM / Springer / arXiv / Otro
URL / DOI	
Tipo de estudio	Revisión sistemática / Estudio empírico / Propuesta de modelo / Caso de estudio / Libro
Dimensión primaria del modelo	D1 / D2 / D3 / D4 / D5
Dimensión secundaria (si aplica)	
Pregunta de investigación que responde	
Aporte principal al corpus	
Limitación identificada	
Brecha que expone	
Métricas o datos cuantitativos	

Campo	Contenido
reportados	
Criterio de inclusión cumplido	Relevancia temática / Período 2020–2025 / Indexación verificada
Notas del revisor	

Ejemplo de Ficha Completada

Campo	Contenido
Código de fuente	[11]
Autor(es)	Midolo, G. et al.
Año de publicación	2024
Título completo	Empirical guidelines for prompt engineering in software development
Fuente / Revista / Conferencia	IEEE Transactions on Software Engineering
Base de datos de indexación	IEEE Xplore
Tipo de estudio	Estudio empírico
Dimensión primaria del modelo	D1
Aporte principal al corpus	10 guías empíricas derivadas del análisis de 4.5 millones de prompts reales

Campo	Contenido
Limitación identificada	Las guías son genéricas, no organizadas por fase del SDLC
Brecha que expone	Ausencia de sistema de diseño contextualizado por tipo de tarea de desarrollo
Métricas reportadas	4,500,000 prompts analizados; mejora de pass@1 de 0.61 a 0.79 con guías aplicadas

Anexo B — Matriz de Operacionalización de Variables (versión expandida)

Variable Independiente: Ingeniería del Prompt en el SDLC

Dimensión	Indicador	Instrumento de medición	Escala	Fuente
Diseño del prompt (D1)	Presencia de los 6 elementos estructurales del prompt	Lista de cotejo de anatomía del prompt	Nominal (Sí/No por elemento)	[10][11]
Diseño del prompt (D1)	Técnica de prompting seleccionada según complejidad	Matriz de selección de técnica	Ordinal (Zero-shot / Few-shot / CoT / ToT / Agéntico)	[11][20]
Evaluación y refinamiento (D2)	pass@k del output generado	Evaluación automatizada con benchmark	Razón (0.0 – 1.0)	[22]

Dimensión	Indicador	Instrumento de medición	Escala	Fuente
Evaluación y refinamiento (D2)	Número de iteraciones de refinamiento hasta aceptación	Registro del ciclo D2	Razón (entero ≥ 1)	[27]
Evaluación y refinamiento (D2)	BLEU score para outputs textuales	Evaluación automatizada	Razón (0.0 – 1.0)	[27]
Aplicación por fases (D3)	Cobertura de fases del SDLC con prompt estructurado	Checklist por fase	Razón (fases cubiertas / 5 fases totales)	[7][8]
Automatización agéntica (D4)	Nivel de madurez agéntica alcanzado	Rúbrica de 5 niveles	Ordinal (N1 – N5)	[23][24]
Seguridad y gobernanza (D5)	Mecanismos de seguridad implementados	Checklist de 3 mecanismos	Razón (mecanismos activos / 3)	[9][25]
Seguridad y gobernanza (D5)	Dimensiones de gobernanza activas	Checklist de 4 dimensiones	Razón (dimensiones activas / 4)	[24][25]

Variable Dependiente: Eficiencia del Proceso de Desarrollo de Software

Dimensión	Indicador	Instrumento de medición	Escala	Fuente
Velocidad	Reducción de tiempo por fase (%) respecto a	Registro de horas por fase	Razón (%)	[7][8]

Dimensión	Indicador	Instrumento de medición	Escala	Fuente
	línea base sin IA			
Velocidad	Número de líneas de código generadas por hora de desarrollo	Registro del equipo	Razón (LOC/hora)	[11]
Calidad	Tasa de defectos post-entrega (bugs por KLOC)	Sistema de seguimiento de bugs	Razón (bugs/KLOC)	[9]
Calidad	Cobertura de pruebas automatizadas (%)	Reporte de cobertura PHPUnit/Jest	Razón (%)	[22]
Calidad	Tasa de vulnerabilidades de seguridad detectadas en revisión	Análisis estático de código	Razón (vulnerabilidades/KLOC)	[9]
Sostenibilidad	Índice de deuda técnica acumulada por sprint	SonarQube u herramienta equivalente	Razón (días técnicos)	[13]
Sostenibilidad	Consistencia de métricas de calidad entre sprints (varianza)	Análisis estadístico de serie temporal	Razón (varianza entre sprints)	[24]

Anexo C — Tabla Maestra de las 28 Referencias del Corpus Documental

La siguiente tabla consolida las 28 fuentes primarias utilizadas en la investigación, organizadas en orden de primera aparición en el texto, conforme al sistema de citación ISO 690 numérico.

N.º	Autor(es)	Año	Título (abreviado)	Tipo	Dimensión	Base de datos
[1]	Pressman, R. S.	2021	Ingeniería del Software: Un enfoque práctico	Libro	D3 base	—
[2]	Sommerville, I.	2020	Software Engineering (10. ^a ed.)	Libro	D3 base	—
[3]	Schwaber, K.; Sutherland, J.	2020	The Scrum Guide	Guía oficial	D3	scrum.org
[4]	Beck, K. et al.	2021	Manifiesto for Agile Software Development	Declaración	D3	agilemanifesto.org
[5]	Royce, W. W.	2020	Managing the Development of Large Software Systems	Clásico	D3 base	IEEE
[6]	Boehm, B.	2020	A Spiral Model of Software Development and Enhancement	Clásico	D3 base	IEEE
[7]	Hou, X. et al.	2024	Large Language Models for Software Engineering: A Systematic Literature Review	Revisión sistemática	D3	arXiv
[8]	Zhang, J. et al.	2024	Unifying the Perspectives of NLP and Software Engineering: A Survey on LLMs	Revisión sistemática	D3	arXiv
[9]	Pearce, H. et al.	2023	Asleep at the Keyboard? Assessing the Security of GitHub	Estudio empírico	D5	IEEE

N.º	Autor(es)	Año	Título (abreviado)	Tipo	Dimensión	Base de datos
			Copilot's Code			
[10]	White, J. et al.	2023	A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT	Propuesta de modelo	D1	arXiv
[11]	Midolo, G. et al.	2024	Empirical guidelines for prompt engineering in software development	Estudio empírico	D1	IEEE
[12]	Hymel, S.	2024	The V-Bounce Model: Redefining Software Development in the Age of AI	Propuesta de modelo	D3/D4	arXiv
[13]	Raja SP	2024	AI-DLC: An AI-Native Development Lifecycle Framework	Propuesta de modelo	D3/D4	Springer
[14]	Brown, T. et al.	2020	Language Models are Few-Shot Learners (GPT-3)	Estudio empírico	D1 base	arXiv
[15]	Rivas Verastegui, J. et al.	2024	Adopción de IA en el desarrollo de software en Perú	Estudio empírico	D3/D5	Scopus
[16]	Tirado Ruiz, M.	2023	GitHub Copilot en entornos universitarios latinoamericanos	Estudio de caso	D3	IEEE
[17]	Torres Villanueva, R.	2024	Brecha de capacitación en IA generativa en equipos de desarrollo junior	Estudio empírico	D1/D5	Scopus

N.º	Autor(es)	Año	Título (abreviado)	Tipo	Dimensión	Base de datos
[18]	Wei, J. et al.	2022	Chain-of-Thought Prompting Elicits Reasoning in Large Language Models	Estudio empírico	D1	arXiv
[19]	Yao, S. et al.	2023	Tree of Thoughts: Deliberate Problem Solving with LLMs	Propuesta de modelo	D1	arXiv
[20]	Liu, P. et al.	2023	Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods	Revisión sistemática	D1	ACM
[21]	Vaswani, A. et al.	2023	Attention Is All You Need	Estudio empírico	Base teórica	arXiv
[22]	Chen, M. et al.	2023	Evaluating Large Language Models Trained on Code (Codex / pass@k)	Estudio empírico	D2	arXiv
[23]	Wang, L. et al.	2024	A Survey on Large Language Model based Autonomous Agents	Revisión sistemática	D4	arXiv
[24]	Rasheed, Z. et al.	2024	AI Agents in Software Engineering: Autonomy, Governance and MLOps	Propuesta de modelo	D4/D5	Springer
[25]	OWASP	2024	OWASP Top 10 for Large Language Model Applications	Guía técnica	D5	owasp.org

N.º	Autor(es)	Año	Título (abreviado)	Tipo	Dimensión	Base de datos
[26]	Tetteh, E.	2024	Iterative Prompt Refinement Strategies for Code Generation Tasks	Estudio empírico	D2	IEEE
[27]	Jiang, X. et al.	2025	Prompt Versioning and Quality Management in LLM-Assisted Development	Propuesta de modelo	D2	arXiv
[28]	Amaro, R. et al.	2025	Systematic Integration of Generative AI Across SDLC Phases	Revisión sistemática	D3	Scopus

Anexo D — Catálogo de Prompts del Modelo Prompt-Centered SDLC

El presente catálogo consolida los prompts estructurados desarrollados y utilizados en la Intervención Metodológica (sección IM.3), organizados por fase del SDLC para facilitar su reutilización y adaptación.

Estructura de cada entrada del catálogo

Campo	Descripción
Código	Identificador único (Letra de fase + número secuencial)
Fase del SDLC	Requisitos / Diseño / Implementación / Testing

Campo	Descripción
	/ Mantenimiento
Técnica	Zero-shot / Few-shot / CoT / ToT / Agéntico
Dimensiones activadas	D1 a D5 según corresponda
Propósito	Descripción de una línea del objetivo del prompt
Elementos estructurales presentes	Rol / Contexto / Tarea / Formato / Restricciones / Ejemplos
Métricas de evaluación D2	Métrica aplicada y umbral
Versión	v1.0 (original) o versión refinada

Resumen del Catálogo

Código	Fase	Técnica	Propósito	Versión
R-01	Requisitos	CoT	Elicitación y estructuración de requisitos desde descripción informal	v1.0
R-02	Requisitos	Few-shot	Generación de criterios de aceptación en formato Gherkin	v1.0
D-01	Diseño	Few-shot	Diseño del esquema relacional de base de datos en PostgreSQL	v1.0
D-02	Diseño	ToT	Evaluación comparativa de alternativas de arquitectura	v1.0

Código	Fase	Técnica	Propósito	Versión
			Laravel	
I-01	Implementación	Few-shot	Generación del modelo Eloquent completo con relaciones y scopes	v1.1 (refinado)
I-01b	Implementación	Zero-shot	Corrección de violaciones PSR-12 en modelo Eloquent	v1.0
I-02	Implementación	Few-shot	Generación del controlador RESTful con Repository Pattern	v1.0
T-01	Testing	Few-shot	Generación de tests unitarios PHPUnit con 6 escenarios de borde	v1.1 (refinado)
T-01b	Testing	CoT	Diagnóstico y corrección de test fallido + método Repository faltante	v1.0
T-02	Testing	Few-shot	Generación de prueba E2E con Cypress para flujo completo de matrícula	v1.0
M-01	Mantenimiento	CoT	Análisis de impacto de cambio de requisito en sistema en producción	v1.0
M-02	Mantenimiento	Zero-shot	Generación de documentación OpenAPI 3.0 para endpoint modificado	v1.0

Nota de uso: Cada prompt del catálogo está diseñado para el sistema SGAU descrito en la Intervención Metodológica. Para adaptarlos a otro proyecto, reemplazar los valores del bloque **Contexto** manteniendo intacta la estructura de los bloques Rol, Tarea, Formato y Restricciones.

Anexo E — Glosario de Términos Técnicos

Agente de IA (AI Agent): Sistema de software que utiliza un LLM como motor de razonamiento para ejecutar tareas de forma autónoma, tomando decisiones secuenciales y utilizando herramientas externas sin intervención humana paso a paso [23].

BLEU Score (Bilingual Evaluation Understudy): Métrica de evaluación automática que mide la similitud entre un texto generado por un modelo de lenguaje y uno o más textos de referencia escritos por humanos. Se usa en esta investigación para evaluar outputs textuales del LLM como documentación y criterios de aceptación [27].

Chain-of-Thought (CoT): Técnica de prompting que instruye al LLM a razonar paso a paso antes de emitir una respuesta final, mejorando significativamente el desempeño en tareas de razonamiento multi-etapa [18].

Corpus documental: Conjunto de fuentes primarias seleccionadas mediante criterios de inclusión explícitos para servir como base empírica de una investigación de revisión sistemática.

DevOps: Conjunto de prácticas que combinan el desarrollo de software (Dev) con las operaciones de TI (Ops), orientadas a acortar el ciclo de vida del desarrollo y entregar software de alta calidad de forma continua.

Few-shot prompting: Técnica de prompting en la que se incluyen uno o más ejemplos (shots) del output esperado dentro del prompt, para guiar al LLM hacia el formato y estilo deseados [14].

Gobernanza de IA: Marco de políticas, procesos y responsabilidades que asegura que los sistemas de IA operen de forma ética, transparente, trazable y alineada con los objetivos de la organización [24][25].

Ingeniería del Prompt (Prompt Engineering): Disciplina que estudia el diseño, estructuración, evaluación y refinamiento de instrucciones textuales (prompts) para obtener outputs de alta calidad y consistencia de los modelos de lenguaje grande [10][11].

LLM (Large Language Model): Modelo de lenguaje de gran escala entrenado sobre enormes corpus de texto mediante arquitecturas transformer, capaz de generar, resumir, traducir y razonar sobre texto en lenguaje natural y lenguajes de programación [14][21].

MLOps (Machine Learning Operations): Conjunto de prácticas que extienden los principios de DevOps al ciclo de vida de los modelos de machine learning, cubriendo entrenamiento, despliegue, monitoreo y actualización continua [24].

pass@k: Métrica de evaluación de generación de código que mide la probabilidad de que al menos una de k muestras generadas por el LLM para un problema dado sea funcionalmente correcta. pass@1 mide si la primera muestra es correcta; pass@10 si alguna de 10 muestras lo es [22].

Prompt: Instrucción textual estructurada enviada a un modelo de lenguaje grande para elicitación de una respuesta específica. En el contexto del SDLC, actúa como el mecanismo de orquestación entre el desarrollador y el LLM [10].

Prompt Injection: Ataque de seguridad en el que un actor malicioso introduce instrucciones en el input del LLM para manipular su comportamiento y eludir las restricciones definidas por el desarrollador [25].

ROUGE-L (Recall-Oriented Understudy for Gisting Evaluation): Métrica de evaluación de resúmenes y textos generados que mide la subsecuencia común más larga entre el output generado y el texto de referencia [27].

SDLC (Software Development Life Cycle): Marco que describe las fases, actividades y procesos del desarrollo de software desde la concepción hasta el retiro del sistema, incluyendo Requisitos, Diseño, Implementación, Testing y Mantenimiento [1][2].

SWE-BENCH: Benchmark de evaluación de LLMs para tareas de ingeniería de software del mundo real, que mide la capacidad del modelo para resolver issues reales de repositorios GitHub públicos [22].

Tree-of-Thought (ToT): Extensión del Chain-of-Thought que permite al LLM explorar múltiples caminos de razonamiento en paralelo antes de seleccionar la respuesta óptima, especialmente útil para decisiones con múltiples alternativas válidas [19].

Zero-shot prompting: Técnica de prompting en la que el LLM recibe una instrucción sin ejemplos previos, confiando en su conocimiento preentrenado para generar el output esperado. Apropiaada para tareas simples y bien definidas [14].

Anexo F — Diagrama del Modelo Prompt-Centered SDLC

El presente anexo contiene la representación visual del ciclo de las cinco dimensiones del modelo Prompt-Centered SDLC, descrito en la sección 5.4.2 del Capítulo 5.

El diagrama fue generado con la herramienta draw.io utilizando el siguiente código Mermaid:

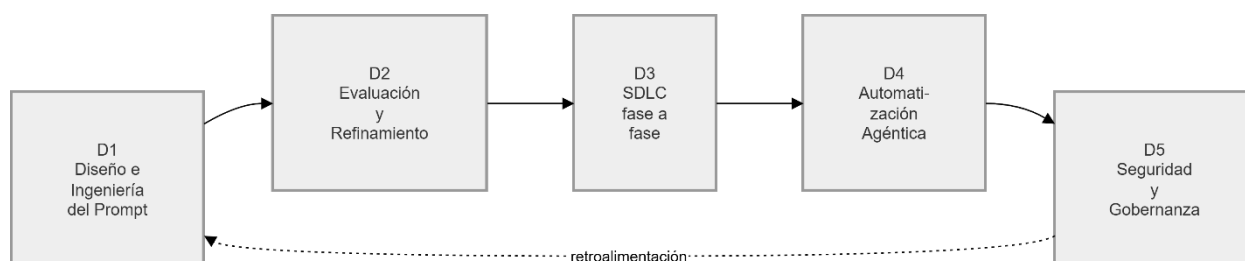


Figura F.1. Arquitectura general del modelo Prompt-Centered SDLC.
El ciclo fluye de D1 a D5 en sentido lineal; la línea punteada representa

el flujo de retroalimentación de gobernanza y monitoreo (D5) hacia el rediseño del sistema de prompts (D1), cerrando el ciclo de mejora continua.

LISTA DE REFERENCIAS

- [1] Michigan Technological University – Oficina de Seguridad y Aseguramiento de TI. *System Development Lifecycle (SDLC)*. Política y Procedimiento Oficial de la Universidad. Revisión 9/20/2016.
- [2] University of South Dakota – Information Technology Services. *ITS Software Development Lifecycle (SDLC) Policy*. Portal oficial USD. 2026.
- [3] YAS, Q. M.; ALAZZAWI, A.; RAHMATULLAH, B. A Comprehensive Review of Software Development Life Cycle Methodologies: Pros, Cons, and Future Directions. *Iraqi Journal for Computer Science and Mathematics*, 2023, vol. 4, n.º 4, art. 14, pp. 173–190.
- [4] ALERYANI, R.; ALSABRY, A.; ALRAMADA, E. Systematic Review of Agile Development Methodologies: Practices, Benefits, Implementation Challenges, and Future Directions. *Sana'a University Journal of Applied Sciences and Technology*, 2025, vol. 3, n.º 6, pp. 1429–1451.
- [5] PADMANABAN, P. C. H.; SHARMA, Y. K. Implication of Artificial Intelligence in Software Development Life Cycle: A State of the Art Review. *International Journal of Recent Research Aspects (IJRRA)*, 2019, vol. 6, n.º 2, pp. 93–98.

- [6] NAVAEI, M.; TABRIZI, N. Machine Learning in Software Development Life Cycle: A Comprehensive Review. En: *Proceedings of the 17th International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE 2022)*. 2022, pp. 344–354.
- [7] HOU, X.; ZHAO, Y.; LIU, Y.; YANG, Z.; WANG, K.; LI, L.; LUO, X.; LO, D.; GRUNDY, J.; WANG, H. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 2024, art. 1, pp. 1–79.
- [8] ZHANG, Q.; FANG, C.; XIE, Y.; ZHANG, Y.; YANG, Y.; SUN, W.; YU, S.; CHEN, Z. A Survey on Large Language Models for Software Engineering. *arXiv preprint (cs.SE)*, 2024, pp. 1–61.
- [9] ABDALHAMID, S.; ALMABRUK, T. Transforming Software Development: A Comparative Study of Traditional and AI-Integrated SDLC Approaches. *American Journal of Engineering Research (AJER)*, 2025, vol. 14, n.º 9, pp. 07–13.
- [10] SHRIVASTAVA, S.; ARORA, K.; DUBEY, A.; THAKUR, D.; SAHAY, S. *Generative AI for Software Developers*. 1.^a ed. BitMaple/Packt, 2025.
- [11] MIDOLO, A.; GIAGNORIO, A.; ZAMPETTI, F.; TUFANO, R.; BAVOTA, G.; DI PENTA, M. Guidelines to Prompt Large Language Models for Code Generation: An Empirical Characterization. *arXiv preprint (cs.SE)*, 2026, pp. 1–12.

- [12] HYMEL, C. *The AI-Native Software Development Lifecycle: A Theoretical and Practical New Methodology*. White Paper. Crowdbotics, 2024.
- [13] RAJA, S. P. *AI-Driven Development Lifecycle (AI-DLC) Method Definition*. White Paper técnico. Amazon Web Services, 2024.
- [14] PALADINO, M.; PONS, C. From Obstacles to Strategies: Integrating Artificial Intelligence into the Software Development Life Cycle. En: *Memorias del 54 JAIIO – ASSE (Argentine Symposium on Software Engineering)*. 2025, pp. 101–117.
- [15] RIVAS VERASTEGUI, K. P.; TIRADO RUIZ, E. F.; TORRES VILLANUEVA, M. El Impacto de las IA Generadoras de Código en el Trabajo de los Programadores. *Revista Innovación y Software*, 2025, vol. 6, n.º 1, pp. 55–68.
- [16] IUSZTIN, P.; LABONNE, M. *LLM Engineer's Handbook: Master the Art of Engineering Large Language Models from Concept to Production*. Packt Publishing, 2024. ISBN 978-1-83620-007-9.
- [17] FAN, A.; GOKKAYA, B.; HARMAN, M.; LYUBARSKIY, M.; SENGUPTA, S.; YOO, S.; ZHANG, J. M. Large Language Models for Software Engineering: Survey and Open Problems. *arXiv preprint (cs.SE)*, 2023, pp. 1–23.
- [18] MACHA, Y.; PULICHIKKUNNU, S. K. A Survey of DevOps Practices for Machine Learning and Artificial Intelligence Workflows in Modern Software Development. *ESP Journal of Engineering & Technology Advancements

(ESP JETA)*, 2024, vol. 4, n.º 3, pp. 200–208.

[19] AMARO, R.; PEREIRA, R.; MIRA DA SILVA, M. Mapping DevOps Capabilities to the Software Life Cycle: A Systematic Literature Review. *Information and Software Technology*, 2025, vol. 177, art. 107583.

[20] KRISHNA, A.; MEDA, V. *AI Integration in Software Development and Operations: Transformation Through AI Infusion in DevOps, Testing, and SRE*. Apress, 2024. ISBN 979-8-8688-1043-5.

[21] LI, B.; WU, W.; TANG, Z.; SHI, L.; YANG, J.; LI, J.; YAO, S.; QIAN, C. et al. Prompting Large Language Models to Tackle the Full Software Development Lifecycle: A Case Study. *arXiv preprint (cs.CL)*, 2024, pp. 1–21.

[22] JIMENEZ, C. E.; YANG, J.; WETTIG, A.; YAO, S.; PEI, K.; PRESS, O.; NARASIMHAN, K. SWE-BENCH: Can Language Models Resolve Real-World GitHub Issues? *arXiv preprint (cs.CL) / ICLR 2024*, 2024, pp. 1–52.

[23] DAI, S.-C.; XU, J.; TAO, G. A Comprehensive Study of LLM Secure Code Generation. *arXiv preprint (cs.CR)*, 2025, pp. 1–12.

[24] ALMASHHADANI, M.; MISHRA, A.; YAZICI, A. Software Maintenance Practices Using Agile Methods Towards Cloud Environment: A Systematic Mapping. *Software: Evolution and Process*, 2024, vol. 36, e2698.

[25] HUYEN, C. *AI Engineering: Building Applications with Foundation Models*.

O'Reilly Media, 2025. ISBN 978-1-098-16630-4.

[26] TETTEH, S. G. Empirical Study of Agile Software Development Methodologies: A Comparative Analysis. *Asian Journal of Research in Computer Science*, 2024, vol. 17, n.º 5, pp. 30–42.

[27] JIANG, Y.; ZHANG, Y.; LU, L.; TREUDE, C.; SU, X.; HUANG, S.; WANG, T. Enhancing High-Quality Code Generation in Large Language Models with Comparative Prefix-Tuning. *arXiv preprint (cs.SE)*, 2025, pp. 1–19.

[28] AMARO, R.; PEREIRA, R.; MIRA DA SILVA, M. Mapping DevOps Capabilities to the Software Life Cycle: A Systematic Literature Review. *Information and Software Technology*, 2025, vol. 177, art. 107583.

[29] THE STANDISH GROUP. *CHAOS 2020: Beyond Infinity*. Boston: The Standish Group International, 2020.

[30] BLOCH, M.; BLUMBERG, S.; LAARTZ, J. Delivering large-scale IT projects on time, on budget, and on value. *McKinsey Quarterly* [en línea]. Octubre 2012. Disponible en: <https://www.mckinsey.com/business-functions/mckinsey-digital/our-insights/delivering-large-scale-it-projects-on-time-on-budget-and-on-value>

[31] CONSORTIUM FOR INFORMATION AND SOFTWARE QUALITY (CISQ). *The Cost of Poor Software Quality in the US: A 2022 Report*. Boston: CISQ, 2022. Disponible en: <https://www.globenewswire.com/news-release/2022/12/06/2568427/0/en/CISQ-Publishes-the-Cost-of-Poor-Software-Quality-in-the-US-A-2022-Report.html>

[32] ACQUAINTSOFT. *Software Development Budget Overruns: Facts and Statistics* [en línea]. 2024. Disponible en: <https://acquaintsoft.com/blog/software-development-budget-overruns-facts-statistics>

[33] SUCCESS THROUGH SAFE. *Standish CHAOS Report 2021: Agile vs. Waterfall* [en línea]. 2021. Disponible en: <https://www.successthroughsafe.com/blog-1/2021/11/13/standish-chaos-report-2021>

